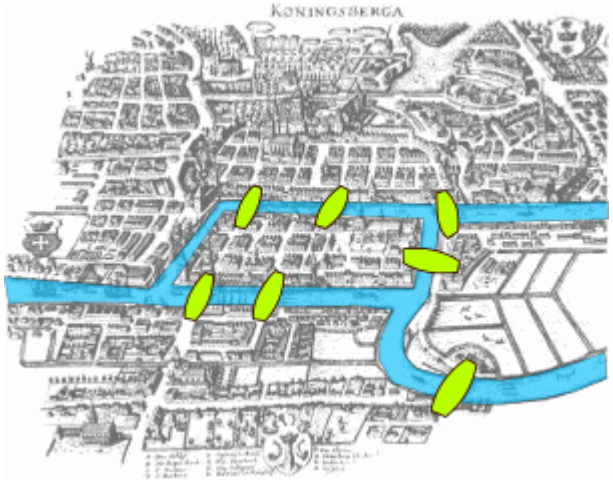


GRAPH-THEORETIC MODELS, Part 1

(download slides and .py files to follow along)

John Guttag

MIT Department Of Electrical Engineering and Computer Science

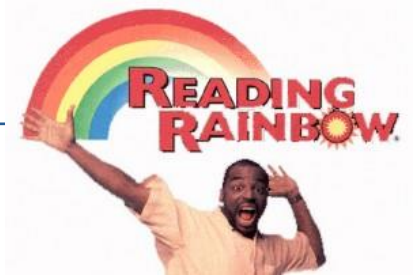
Announcements

- Two lectures this week, both on graphs and graph algorithms
- Monday, 3/16: Optional recitation
- Wednesday, 3/18: Micrco Quiz 3 at 11:05
- Spring Break week of 3/23, no classes or office hours

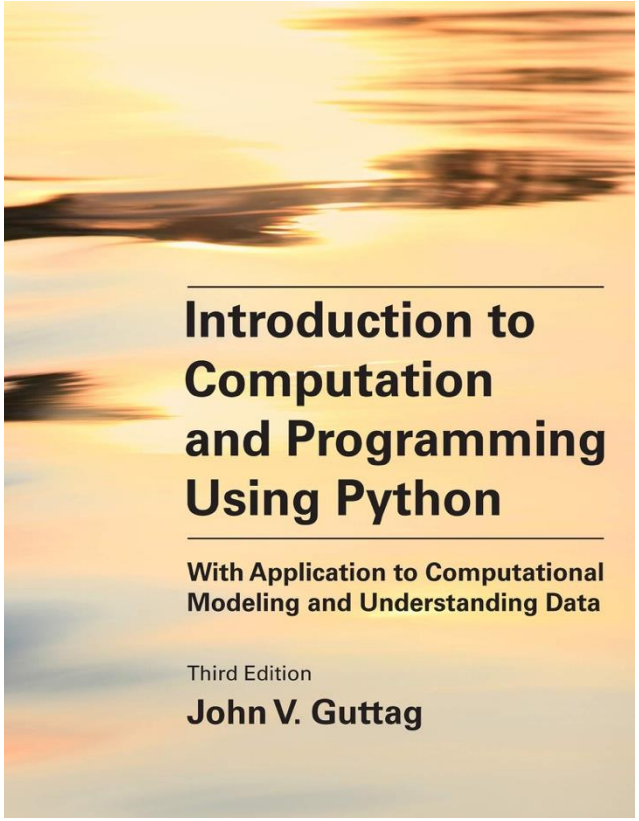
Today

- Graph models and how to implement them
- Shortest path problems on unweighted graphs
- Depth-first search and breadth-first search

Relevant Reading

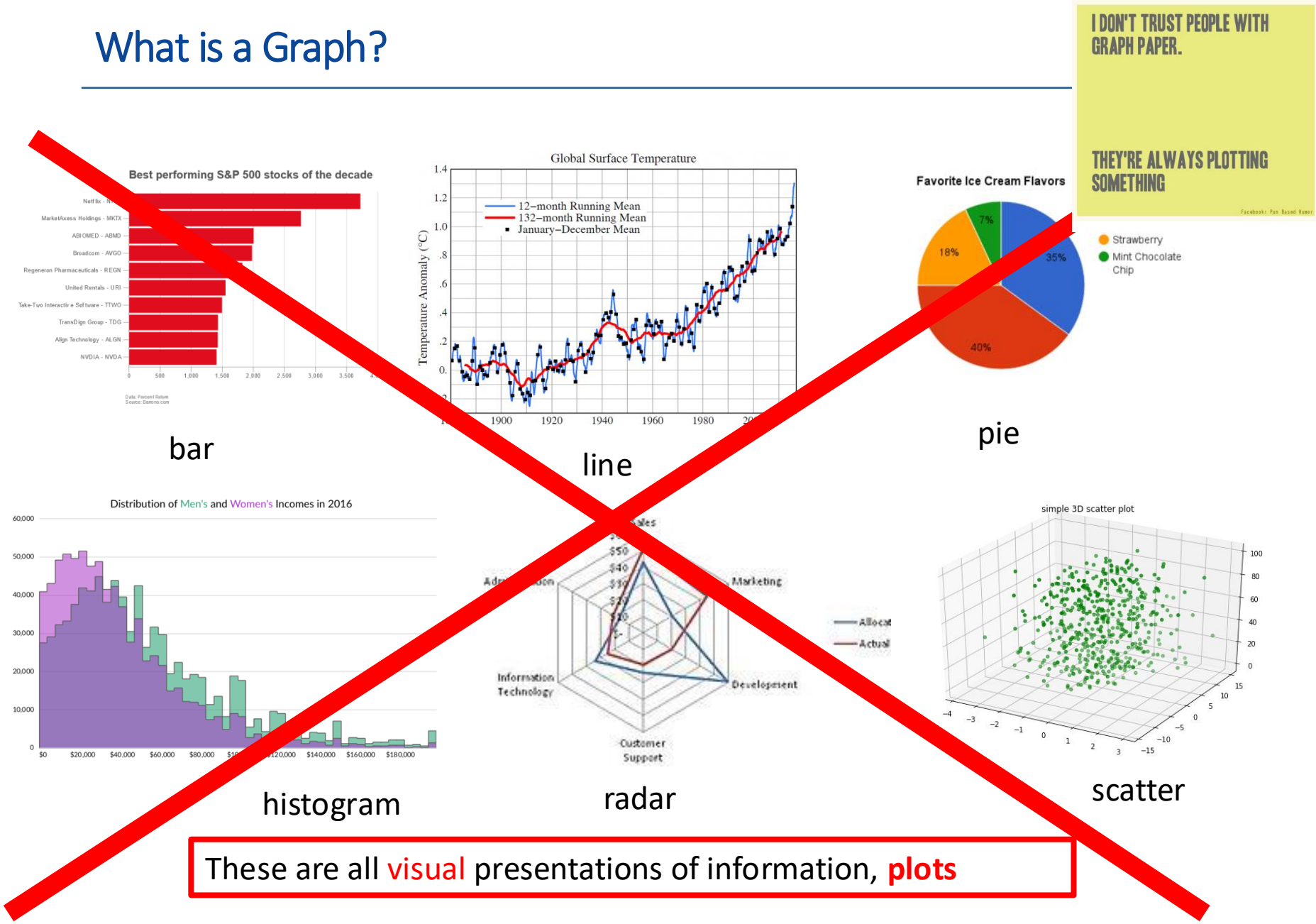


- Today and next Wednesday
  - Section 14.2



All code and an errata sheet at
https://github.com/guttag/Intro-to-Computation-and-Programming

What is a Graph?



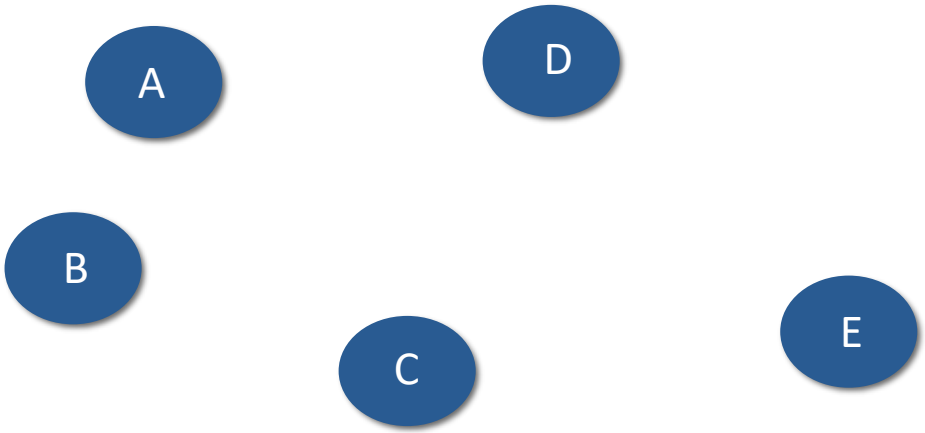
bar line pie

histogram radar scatter

These are all visual presentations of information, **plots**

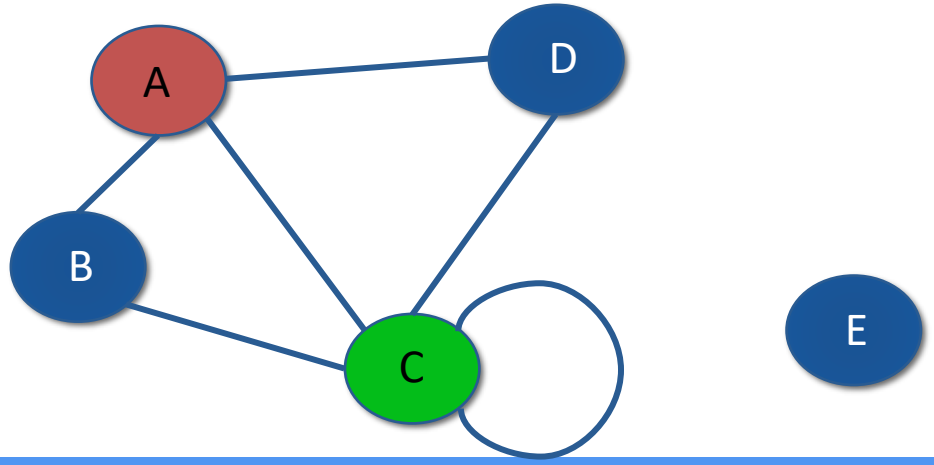
What is a Graph?

- Set of nodes (vertices)

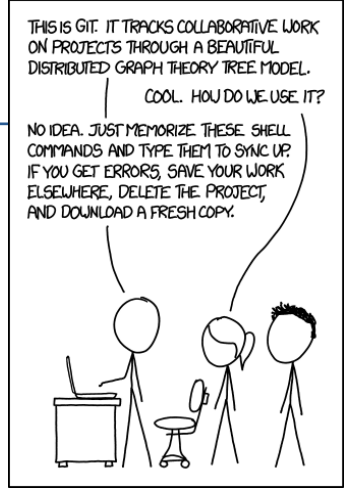


What is a Graph?

- Set of nodes (vertices)
  - Might have associated names or properties
- Set of edges (arcs) each connecting a pair of nodes
  - Undirected (graph)

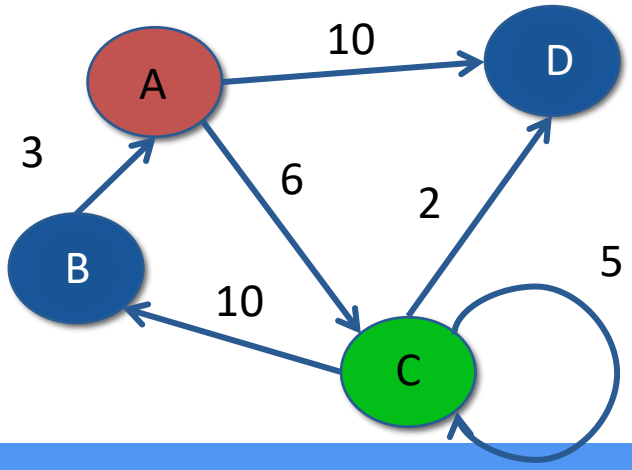


What is a Graph?



- Set of nodes (vertices)
  - Might have properties associated with them
- Set of edges (arcs) each connecting a pair of nodes
  - Undirected (graph)
  - Directed (digraph)
    - Source (parent) and destination (child) nodes
  - Unweighted or weighted
    - Assume weights non-negative

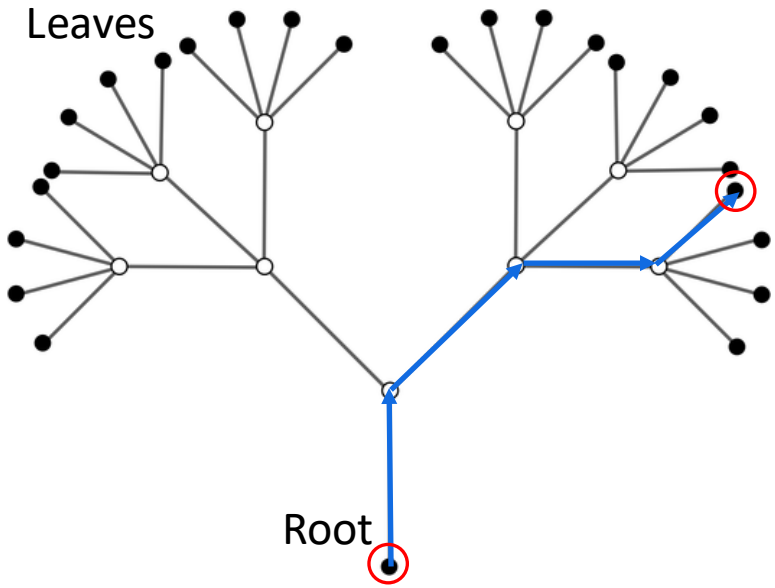
Path: set of edges connecting one node to another, e.g.,
B → A → C → D
B → A → D



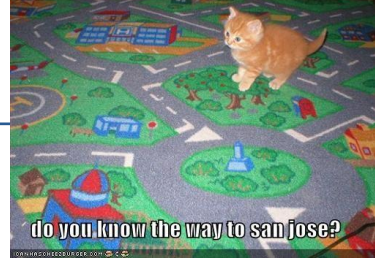
Graph:
- might not be completely connected
- could have loops, both single length and longer

Trees: An Important Special Case

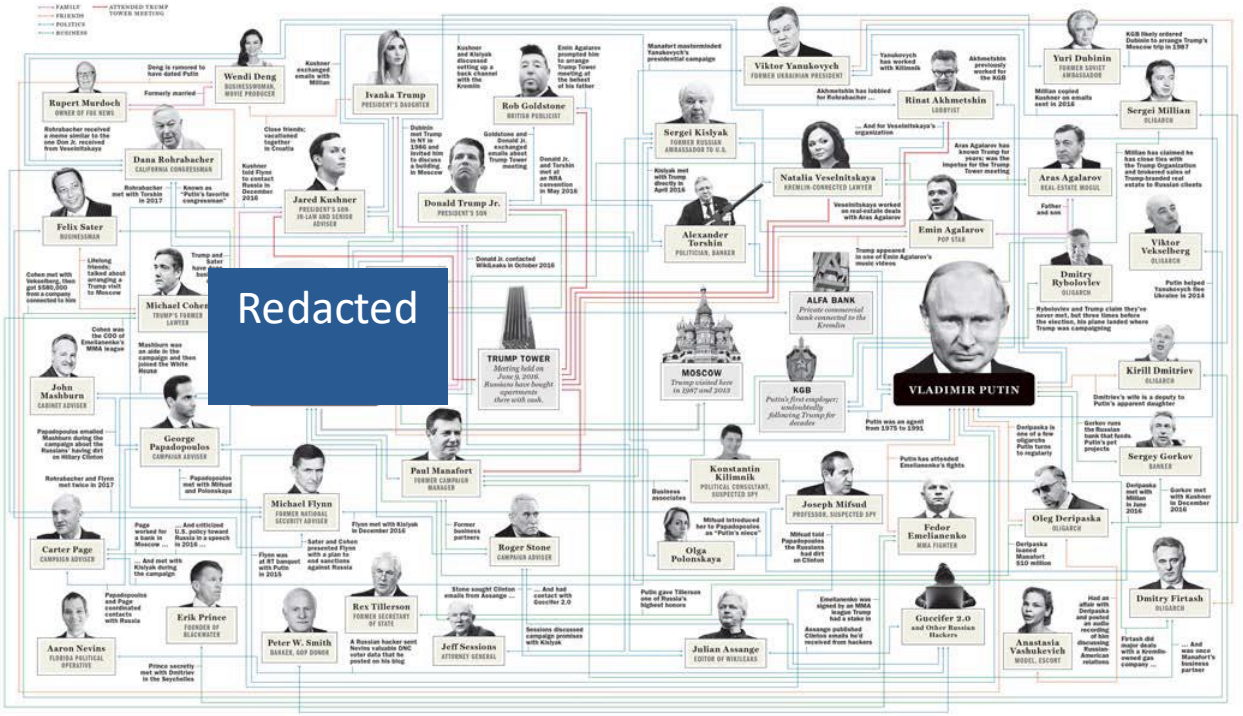
- A special kind of undirected graph in which each pair of nodes is connected by exactly one path
  - Recall the binary search trees we used to solve knapsack problem – we can generalize to more than two branches at a node



Graphs

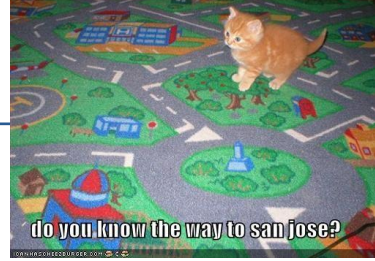


- Capture and reason about relationships among entities
  - Routes between Boston and San Jose
  - How the atoms in a molecule are related to one another
  - Ancestral relationships (family trees)
  - Business/social/political connections
  - …



New York Magazine

Graphs

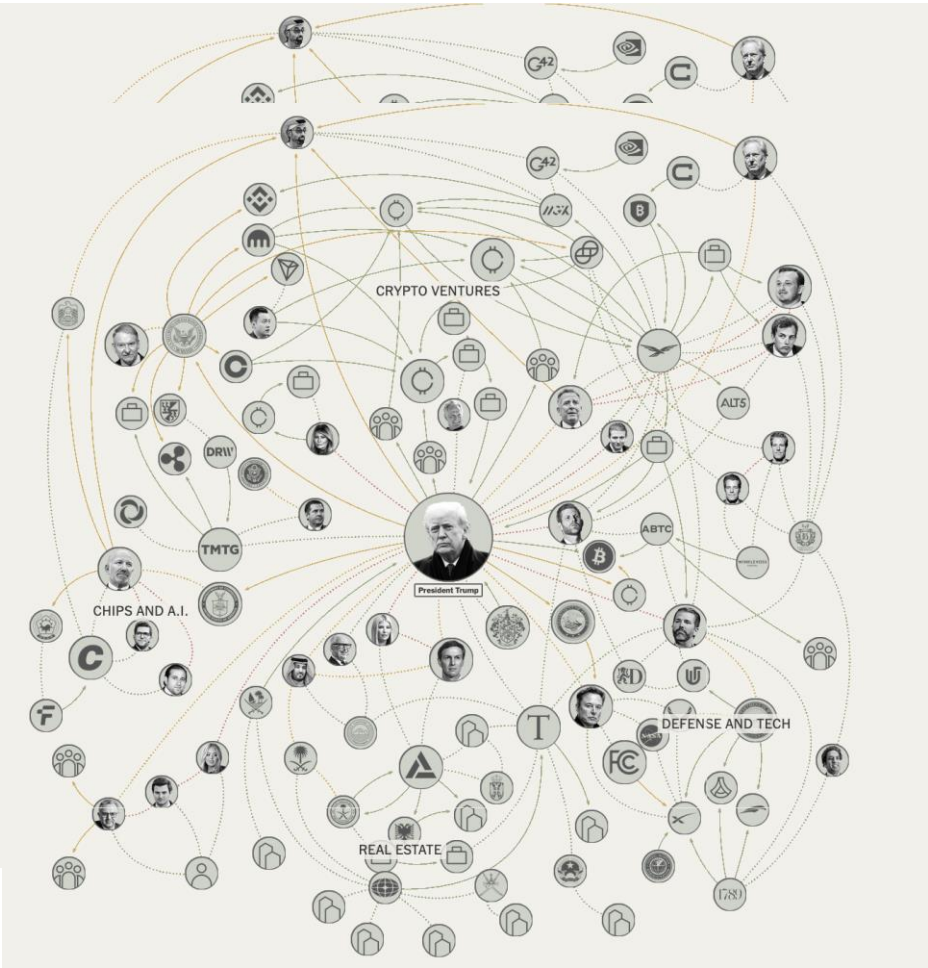


- Capture and reason about relationships among entities
  - Routes between Boston and San Jose
  - How the atoms in a molecule are related to one another
  - Ancestral relationships (family trees)
  - Business/social/political connections
  - …

?

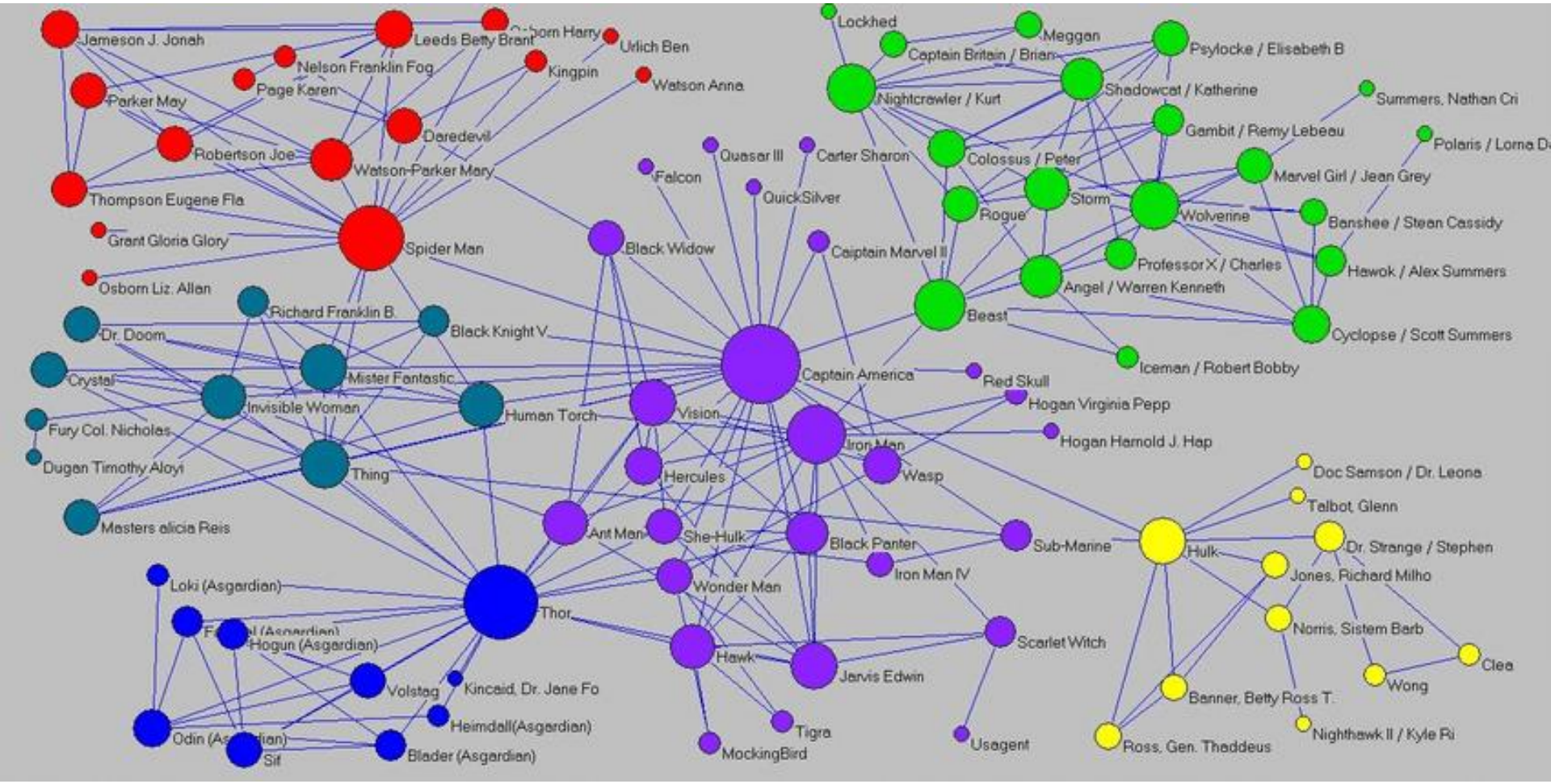
Graphs

- Capture and reason about relationships among entities
  - Routes between Boston and San Jose
  - How the atoms in a molecule are related to one another
  - Ancestral relationships (family trees)
  - Business/social/political connections
  - …



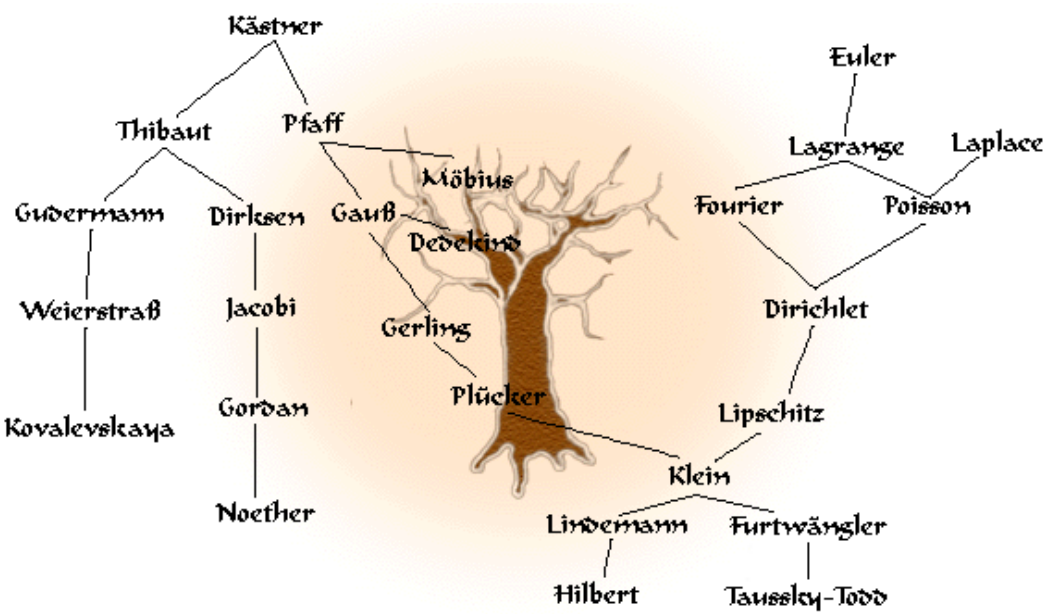
New York Times

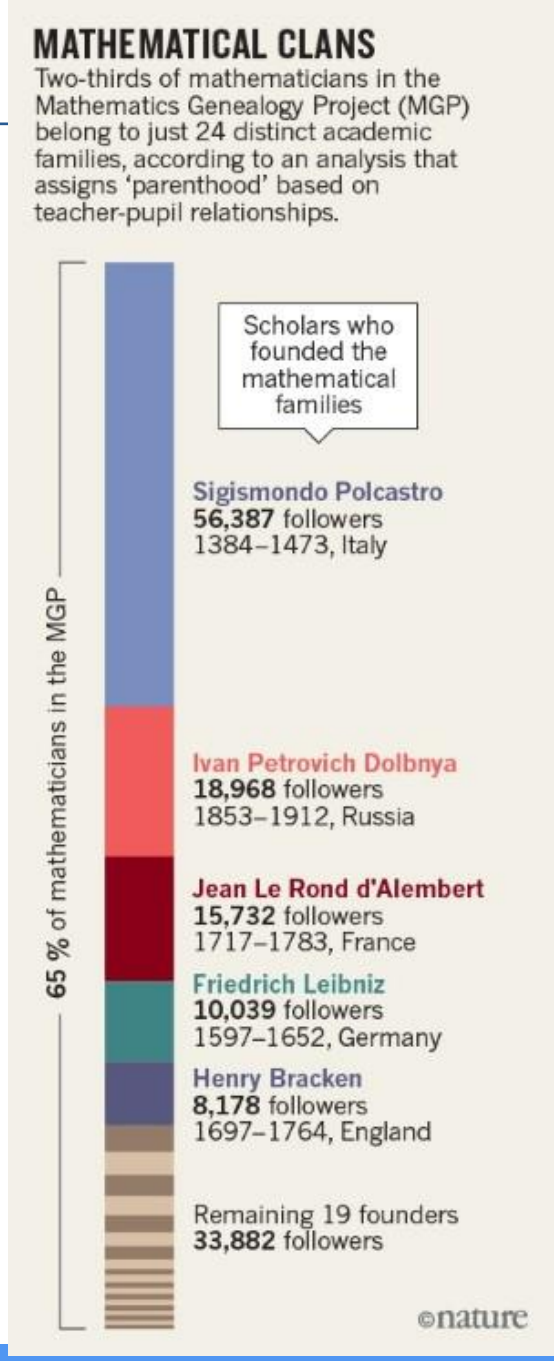
Marvel Cinematic Universe (January 2014)



https://www.researchgate.net/figure/Marvel-heroes-network_fig1_283380787

Graph of History of Mathematics



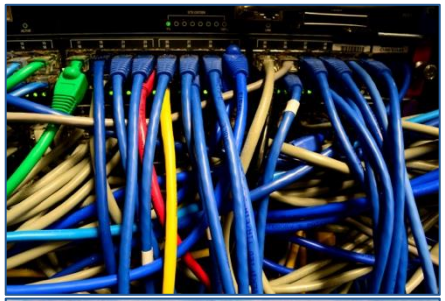


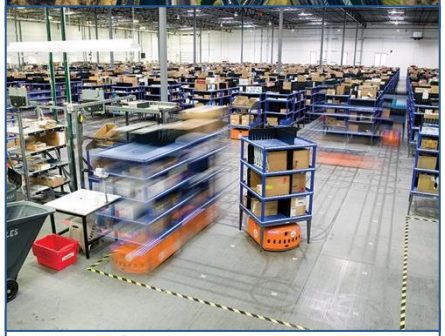
The Path Connecting Leibniz and Me

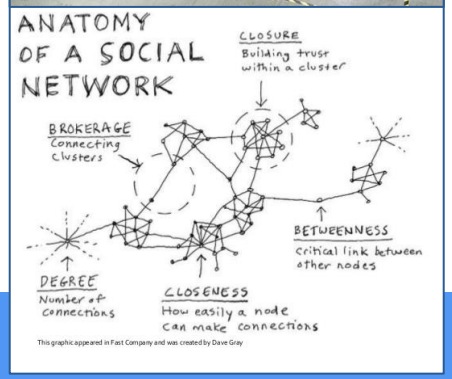
Gottfried Wilhelm Leibniz
Jacob Bernoulli
Johan Bernoulli
Leonhard Euler
Joseph Lagrange
Simeon Poisson
Michel Chasles
H.A. Newton
E.H. Moore
Oswald Veblen
Phillip Franklin
Alan Perlis
Jerome Feldman
James Horning
John Guttag **Regression to the mediocre**

Graphs Model a Wide Range of Systems

- Computer networks
- Transportation networks
- Logistics networks
- Social networks
- Structure of language
- Structure of molecules
- Genomic relationships
- Etc.

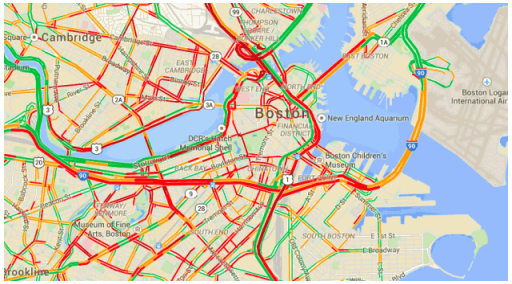


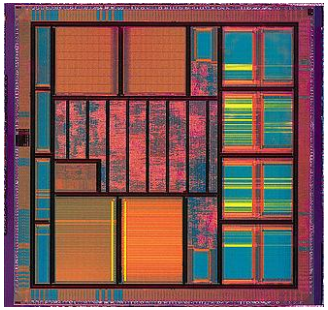




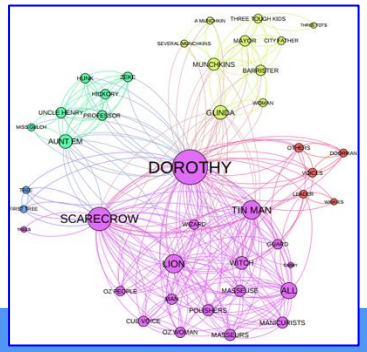
Why Graphs Are So Useful

- Not only do graphs capture relationships in connected networks of items, they support **computation** on those structures
- E.g.,
  - Find least expensive path between elements (aka the **shortest path problem**)
  - Partition graph into k (equal) sized subgraphs with minimal connections between them (aka **graph partition problem** or **graph clique problem**)
  - Find the most efficient way to separate sets of connected elements (aka the **min-cut/max-flow problem**)





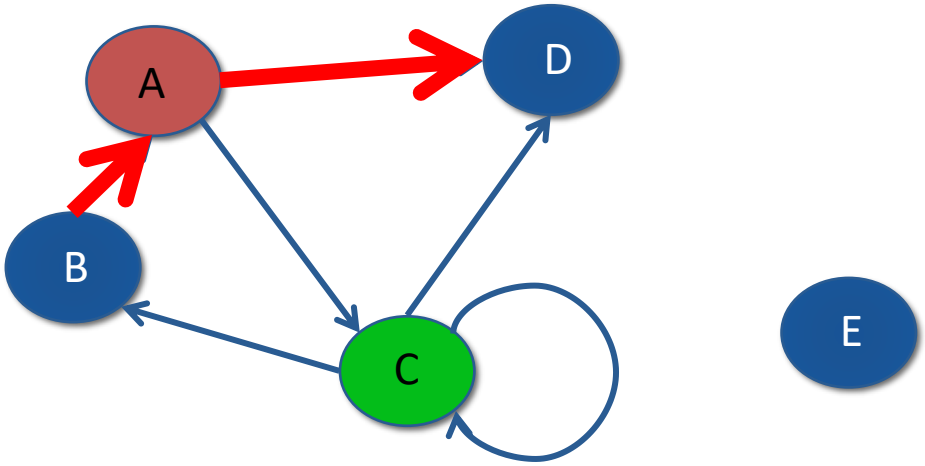
These problems appear in 6.120[6.042], 6.121[6.006], and other classes



Shortest Path Problems

- Shortest (unweighted) path
  - Fewest number of edges from a source node to a destination node

This lecture



Shortest Path Problems

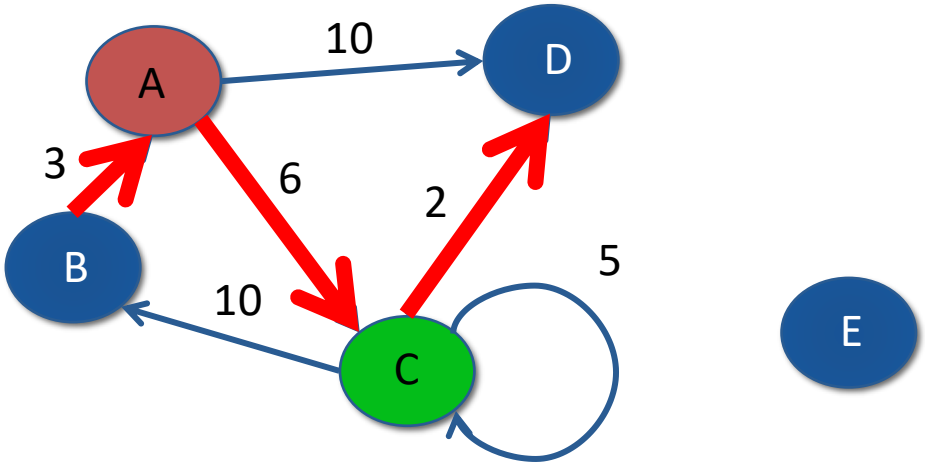
- Shortest (unweighted) path
  - Fewest number of edges from a source node to a destination node

This lecture

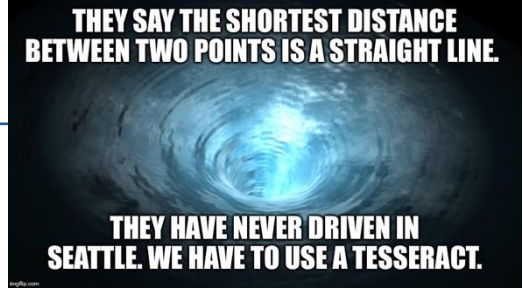
- Shortest weighted path
  - Minimizes the sum of the weights of its edges

Next lecture

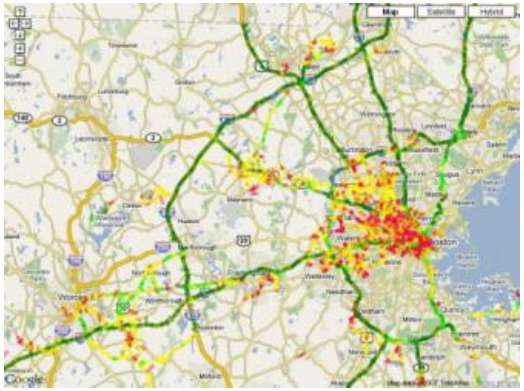
B → A → D
B → A → C → D

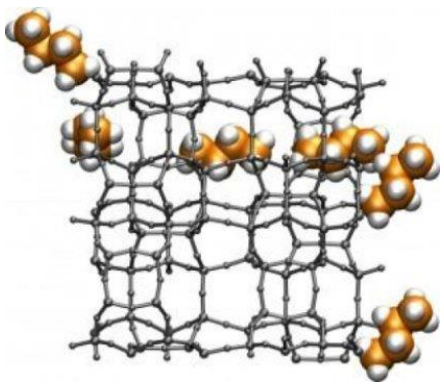


Some Shortest Path Problems



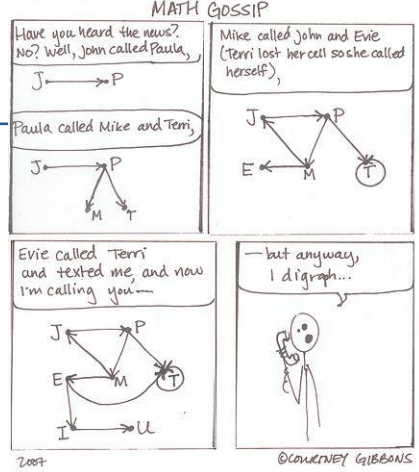
- Finding a route from one city to another
- Routing data on communication networks
- Warehouse logistics of storing and retrieving products
- Finding a path for a molecule through a chemical labyrinth

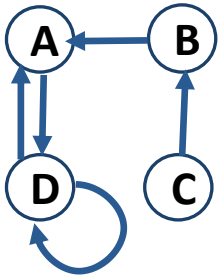




IMPLEMENTING GRAPHS

Representations of Digraphs





- **Digraph is a directed graph**
  - Edges pass in one direction only
  - Need to represent collection of edges
- **Adjacency matrix**
  - Rows: source nodes
  - Columns: destination nodes
  - Cell[s, d] = 1 if there is an edge from s to d
    = 0 otherwise
  - Note that in a digraph, matrix is **not** symmetric
  - Uses **O(|nodes|**2) memory**
  - Easily generalized to weighted edges and to multiple edges between nodes

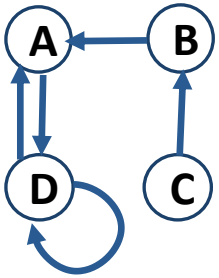
d

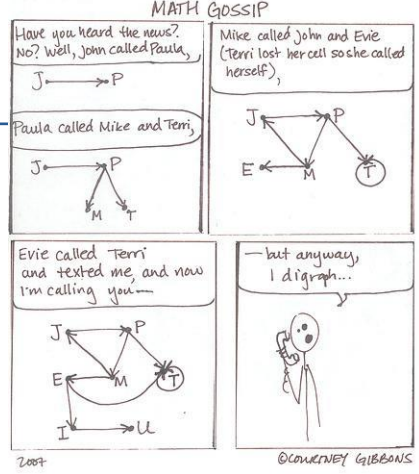
s

	A	B	C	D
A	0	0	0	1
B	1	0	0	0
C	0	1	0	0
D	1	0	0	1

Representations of Digraphs

- Digraph is a directed graph
  - Edges pass in one direction only
  - Need to represent collection of edges
- **Adjacency list**
  - Associate with each node a list of destination nodes that can be reached by one edge
  - Uses **O(|edges|) memory**, therefore good for **sparse** graphs
  - Easily extended to weighted edges





A: [D]
B: [A]
C: [B]
D: [A, D]

Class Digraph, part 1

```
class Digraph:
    """Represents a directed graph using an adjacency list,
       mapping each node to a list of outgoing edges.
       Nodes are strs"""

    def __init__(self, nodes=()):
        self._edges = {}
        for node in nodes:
            self.add_node(node)

    def add_node(self, node):
        if node in self._edges:
            raise ValueError('Duplicate node')
        self._edges[node] = {}

    def add_edge(self, src, dest, weight=1):
        if src not in self._edges:
            self.add_node(src)
        if dest not in self._edges:
            self.add_node(dest)
        self._edges[src][dest] = weight
```

Optionally initialize graph with unconnected nodes

Store nodes implicitly as keys of the dictionary

Add nodes when refenced by new edges

Store edges by assigning edge weight to the destination node

An Example Digraph

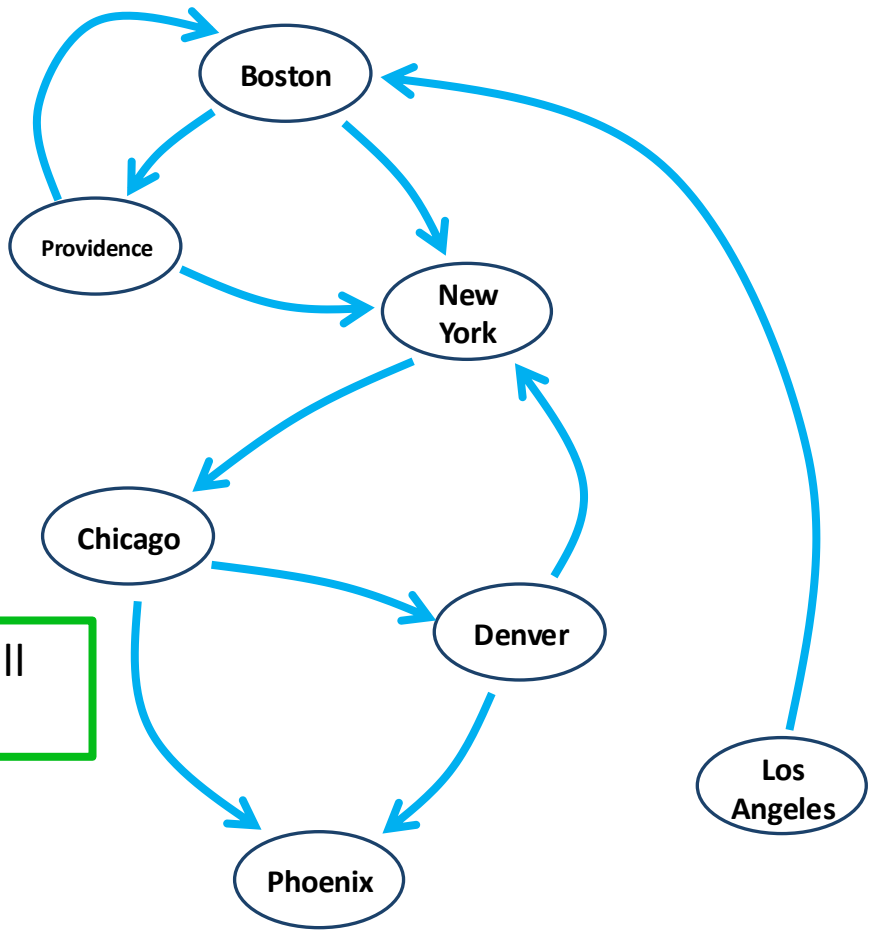
All nodes in graph

	Adjacency List
Boston:	Providence(1), New York(1)
Providence:	Boston(1), New York(1)
New York:	Chicago(1)
Chicago:	Denver(1), Phoenix(1)
Denver:	Phoenix(1), New York(1)
Phoenix:	
Los Angeles:	Boston(1)

Node

Edges from that node

Unweighted edges all have weight 1



{'Boston': {'Providence': 1, 'New York': 1},
'Providence': {'Boston': 1, 'New York': 1},
'New York': {'Chicago': 1}, 'Chicago': {'Denver': 1, 'Phoenix': 1},
'Denver': {'Phoenix': 1, 'New York': 1}, 'Phoenix': {}, 'Los Angeles': {'Boston': 1}}

Class Digraph, part 2

```
def has_node(self, node):
    return node in self._edges

def get_all_nodes(self):
    return list(self._edges.keys())

def outgoing_edges_of(self, node):
    return self._edges[node].copy()

def children_of(self, node):
    return list(self.outgoing_edges_of(node).keys())
```

Don't return an internally-stored dictionary entry, user could mutate

Copy dictionary's keys into list

"Getter" methods

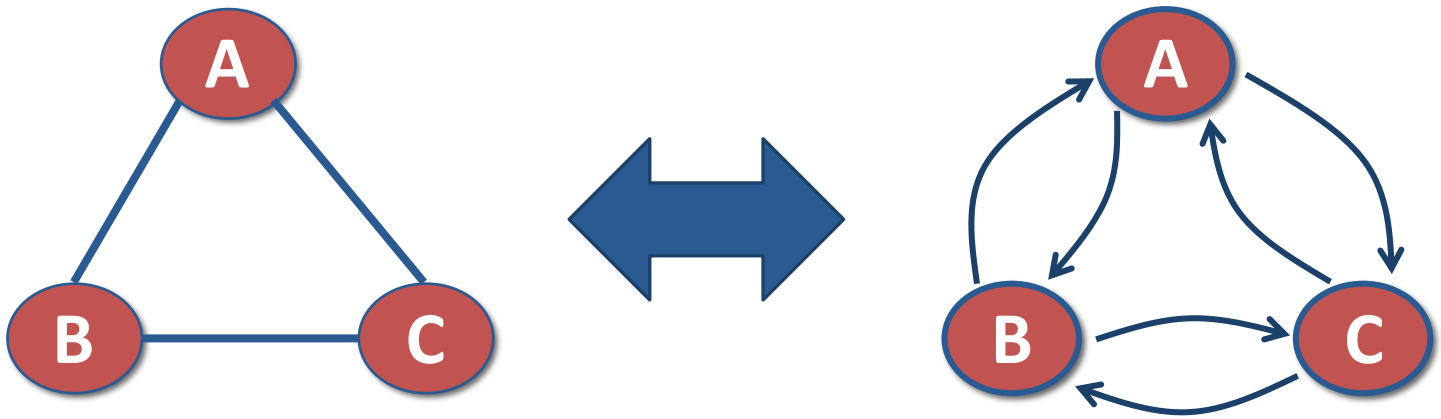
__str__ not shown

Assumptions

- Nodes are **hashable** objects, so can be used as dictionary keys
- **No multi-edges**, at most one edge from any node to another

Implementing Undirected Graphs

- **Observation**: undirected edge can be represented as pair of opposite directed edges



- Undirected graph can **reuse** functionality from `class Digraph`
- Only have to change `add_edge()` method
  - Use **inheritance**, of course

Class Graph (undirected)

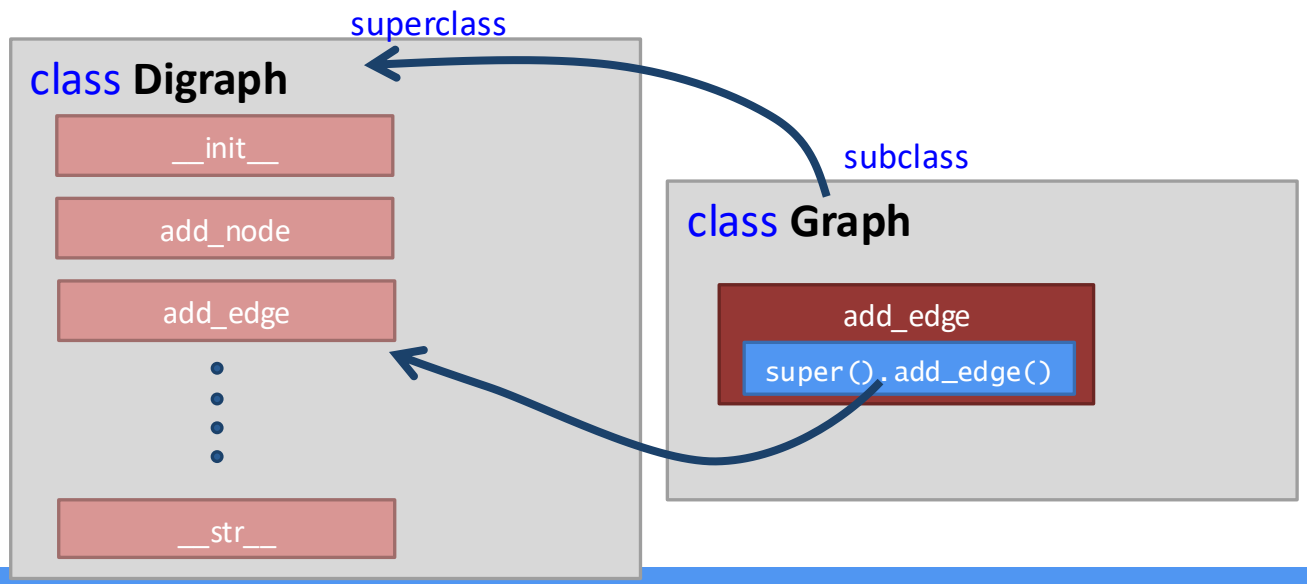
class **Graph** inherits class **Digraph**'s functionality

```
class Graph(Digraph):
    """Represents an undirected graph with pairs
       of directed edges"""

    def add_edge(self, node1, node2, weight=1):
        super().add_edge(node1, node2, weight)
        super().add_edge(node2, node1, weight)
```

Graph's **add_edge** overrides **Digraph**'s

Add both directed edges using existing superclass method

`super()` interprets *`self`* in the context of the parent class



Designing Class Hierarchy

- Why make undirected **`Graph`** a subclass of **`Digraph`**, rather than vice versa?
  - Might seem like **`Digraph`** has additional "feature"
- Follow the **Substitution Principle** (Section 10.2.2 of text):
  - "**Subclass** behavior should be consistent with **superclass**"
    - (See Section 10.2.2 and page 295 of text)

Building and Printing Examples

```python
def build_test_graph(graph_type):
    g = graph_type()
    g.add_edge('A', 'B')
    g.add_edge('A', 'C')
    g.add_edge('A', 'D')
    g.add_edge('B', 'E')
    g.add_edge('C', 'E')
    g.add_edge('C', 'F')
    g.add_edge('C', 'G')
    g.add_edge('E', 'H')
    g.add_edge('F', 'H')
    g.add_edge('G', 'H')
    return g

g = build_test_graph(Digraph)
print('An example directed graph:')
print(g)
print()

g = build_test_graph(Graph)
print('An example undirected graph:')
print(g)
```

```
An example directed graph:
A: B(1), C(1), D(1)
B: E(1)
C: E(1), F(1), G(1)
D:
E: H(1)
F: H(1)
G: H(1)
H:

An example undirected graph:
A: B(1), C(1), D(1)
B: A(1), E(1)
C: A(1), E(1), F(1), G(1)
D: A(1)
E: B(1), C(1), H(1)
F: C(1), H(1)
G: C(1), H(1)
H: E(1), F(1), G(1)
```

SHORTEST-PATH ALGORITHMS (simple)

Finding the Shortest Path

- Algorithm 1, **depth-first search** (DFS)
- Algorithm 2, **breadth-first search** (BFS)

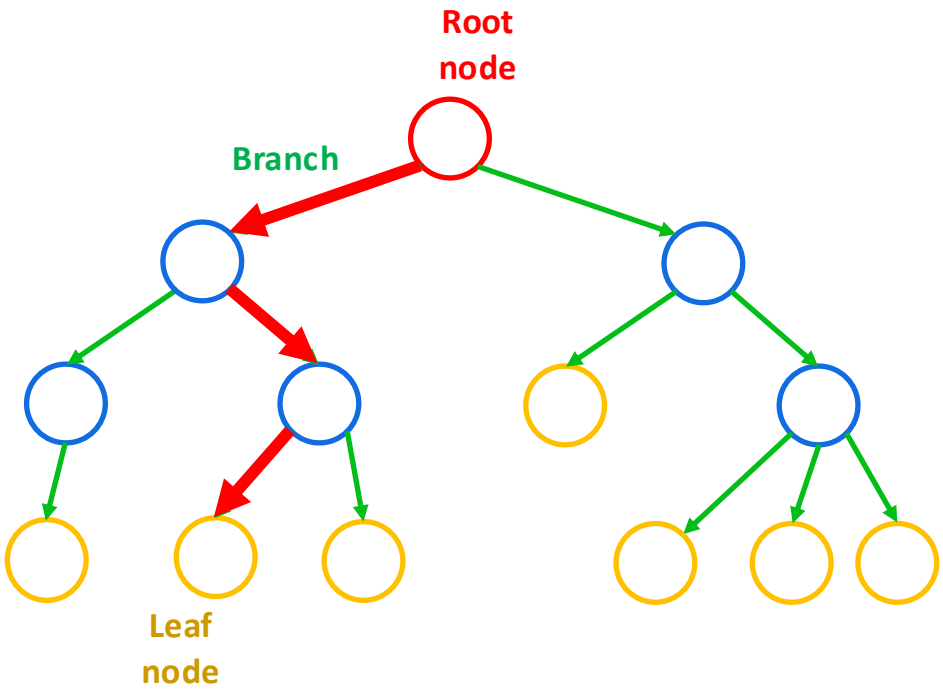
Both use *divide-and-conquer*: if we can find a path from a source to an intermediate node, and a path from the intermediate node to the destination, the combination is a path (*but not necessarily the shortest*) from source to destination

Finding the Shortest Path

- We will iteratively develop the principles of **depth-first search (DFS)** and **breadth-first search (BFS)**
- **Iteration 1:** Finding paths in **trees**
- **Iteration 2:** Generalizing to find shortest paths in **graphs**
- **Iteration 3:** **Unify** the algorithm design of BFS and DFS

Searching for Nodes in Trees

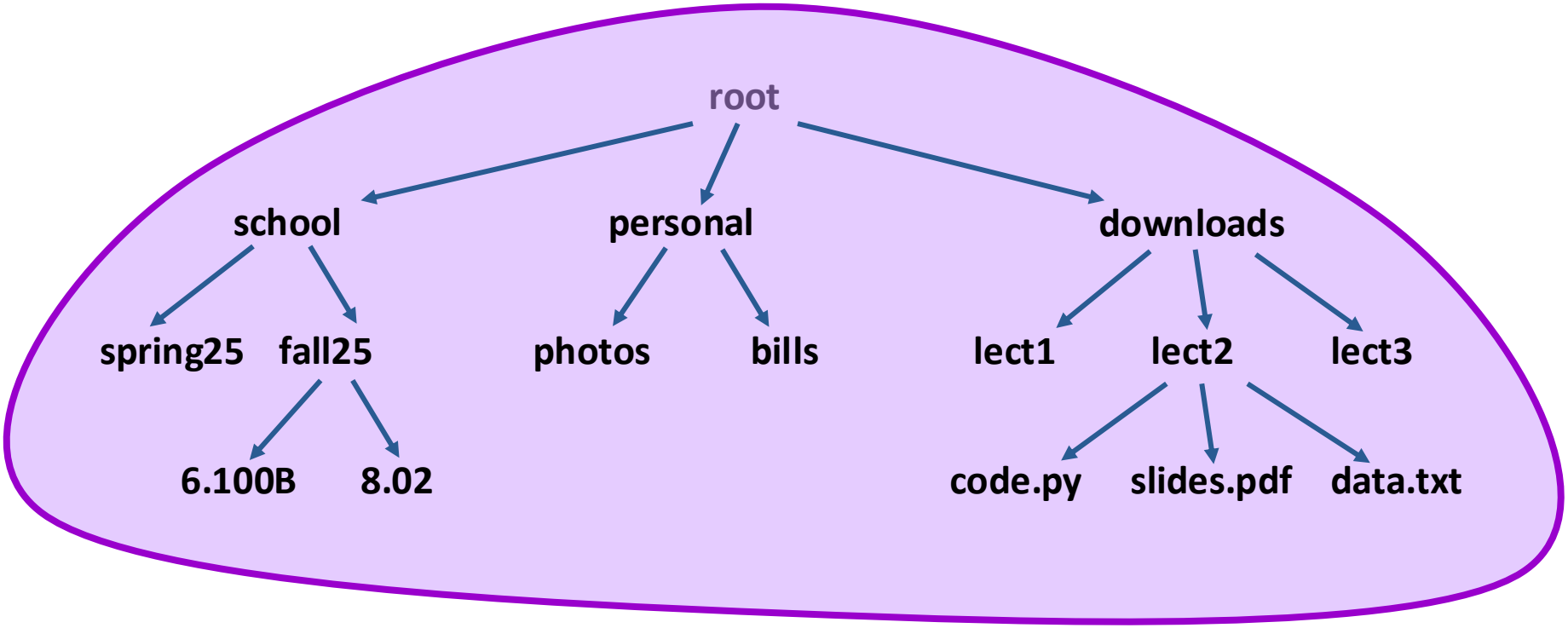
- Recall solution to knapsack problem
- For a tree: **unique path** from root node to any other node
  - Is this true only for root?
- Key question: **In what order should we explore branches?**



An Example Tree

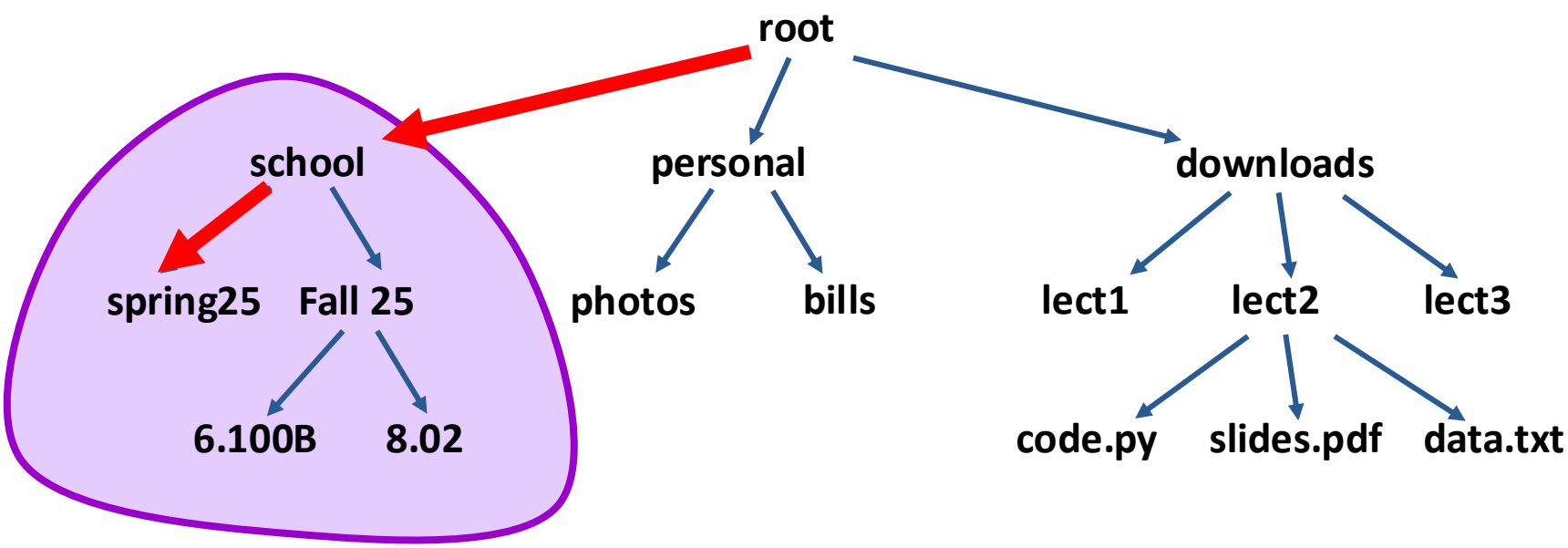
Filesystem is a tree

Find 6.100B Folder



Approach 1: Leverage Recursive Structure

- Adapt our strategy for exploring the knapsack decision tree
- After branching on child, recursively find a path to target, using **child as root of a new tree**



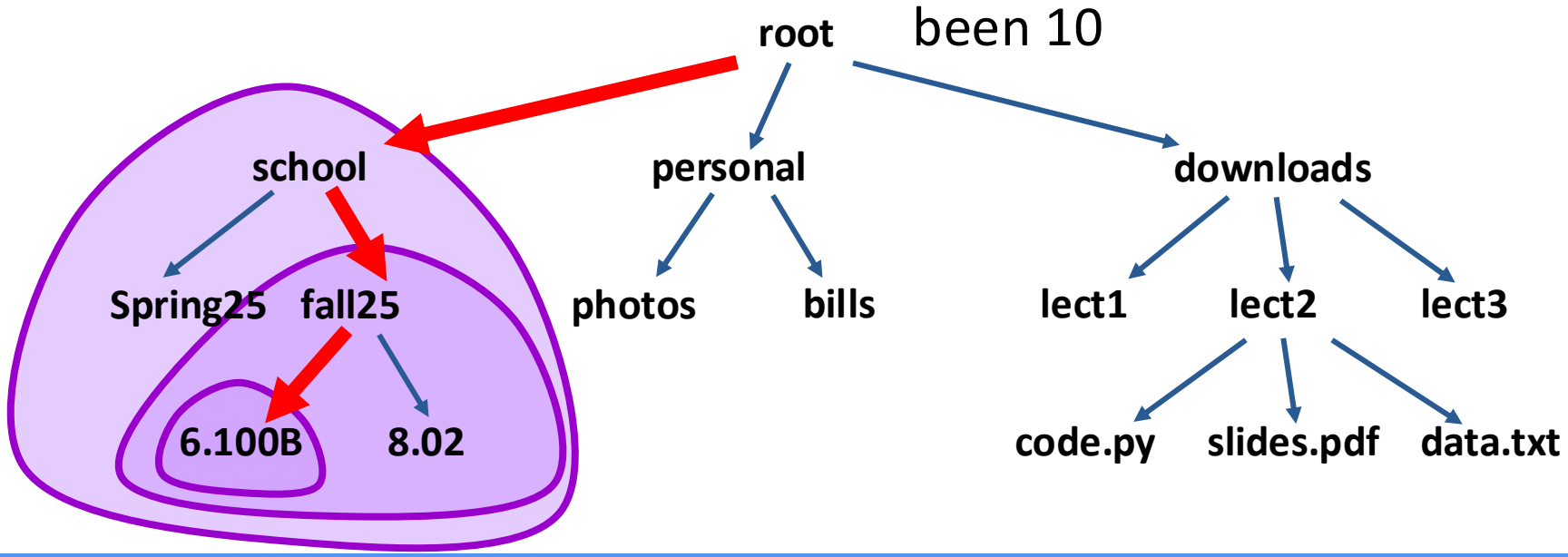
Approach 1: Leverage Recursive Structure

- Adapt our strategy for exploring the knapsack decision tree
- After branching on child, recursively find a path to target, using **child as new root**

Followed 4 links to find 6.100B

If we had searched for downloads, it would have been 10

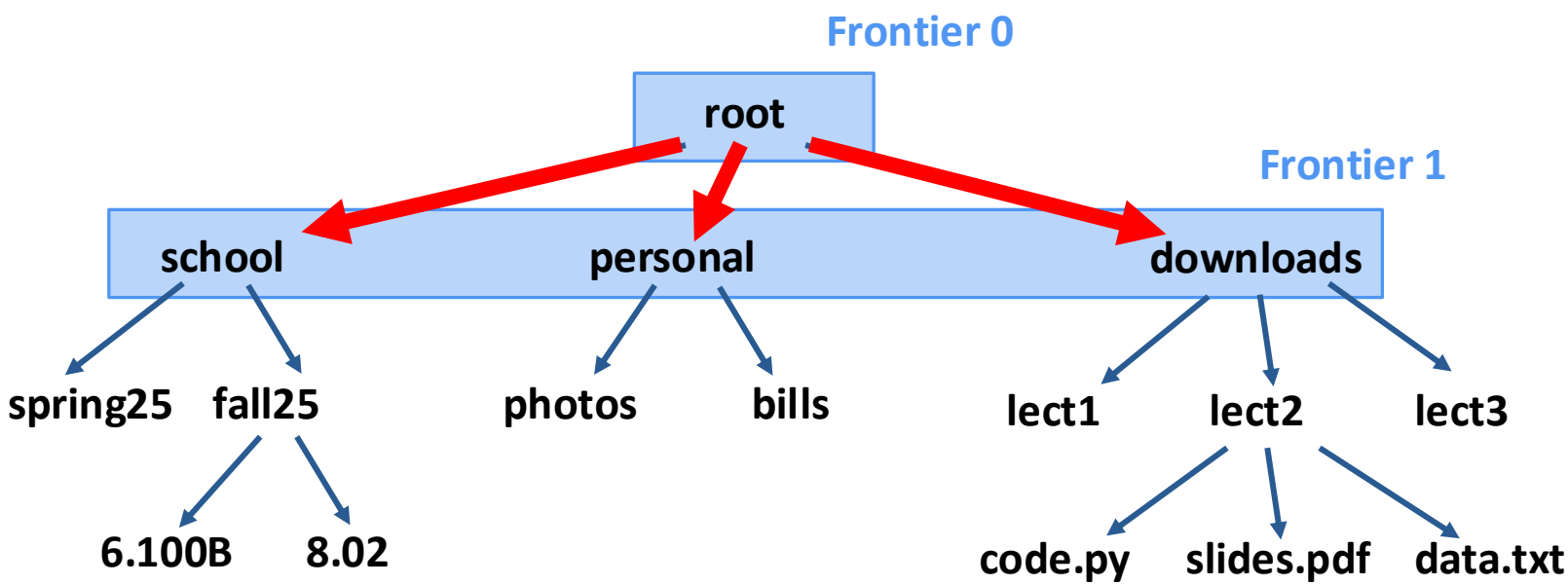
Depth-First Search (DFS)



Approach 2: Breadth First Search

- Follow **successive layers** of depth away from the root
  - Layers are **"frontiers"**
- Each frontier is **one step away** from the previous frontier

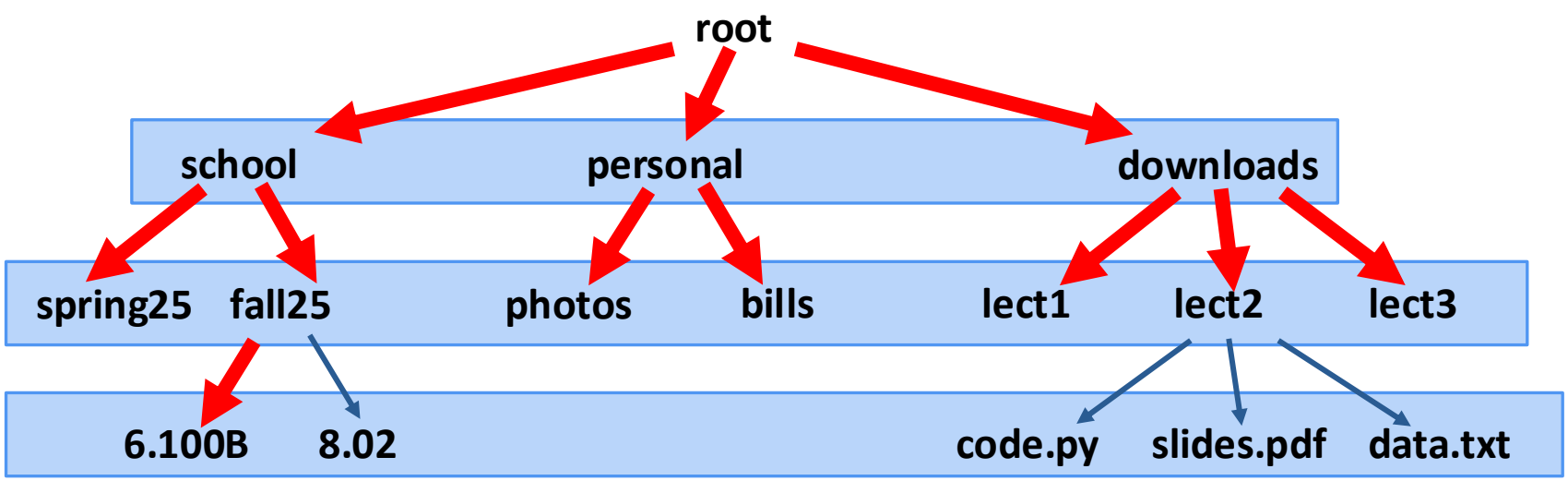
Followed 3 links to find downloads



Approach 2: Breadth First Search

- Follow **successive layers** of depth away from the root
  - Layers are **"frontiers"**
- Each frontier is **one step away** from the previous frontier

Followed 11 links to find 6.100B



DFS on Trees (posted code has a verbose option)

```
def dfs_tree(tree, root, target):

    def dfs_internal(path):

        last_node = path[-1]
        if last_node == target:
            return path

        for next_node in tree.children_of(last_node):
            new_path = dfs_internal(path + [next_node])

            if new_path:
                return new_path
        return None

    return dfs_internal([root])
```

Check if DFS reached the target node. If so return path

Otherwise expand end of path so far to its descendents

Pass solution up the call stack, if path found. Otherwise pass None.

No path.

initialize internal recursive function with path of length 1, starting at root

BFS on Trees (posted code has a verbose option)

```
def bfs_tree(tree, root, target):
    current_frontier = [[root]]
    next_frontier = []

    while current_frontier:

        for path in current_frontier:
            last_node = path[-1]
            for next_node in tree.children_of(last_node):
                new_path = path + [next_node]
                if next_node == target:
                    return new_path
                next_frontier.append(new_path)

        current_frontier, next_frontier = next_frontier, []

    return None
```

Each frontier is a list of paths from the root into the nodes in that frontier

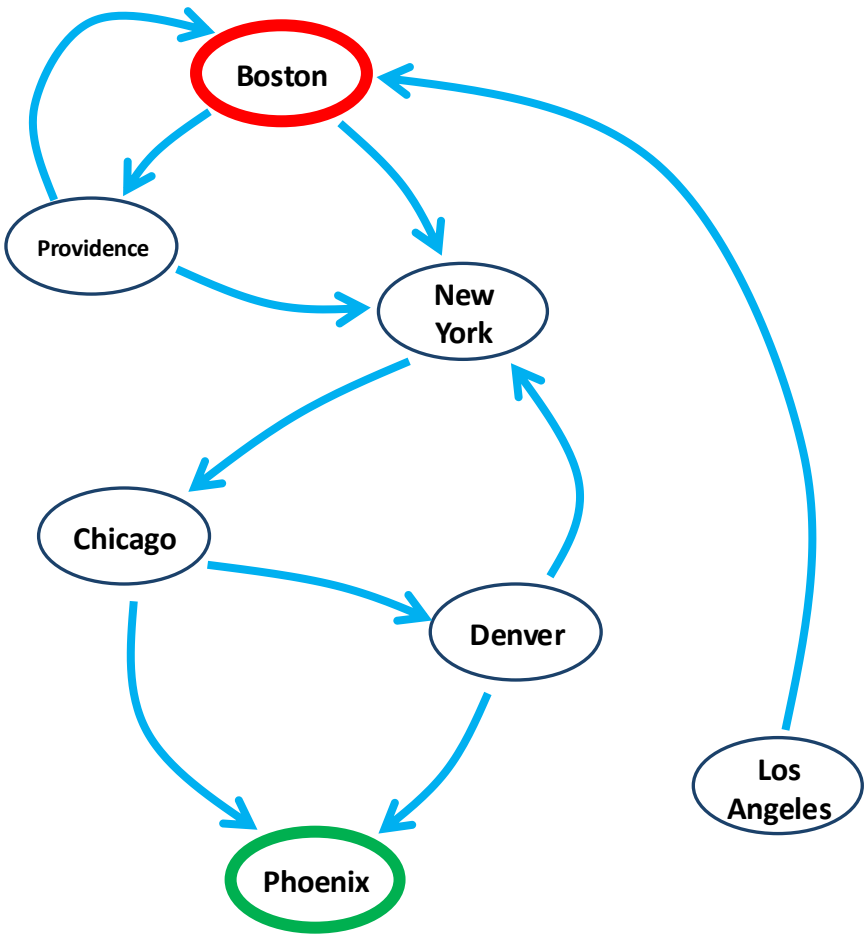
Append all children of current frontier nodes into next frontier

Stop search once we reach target

Slide frontier window down a level

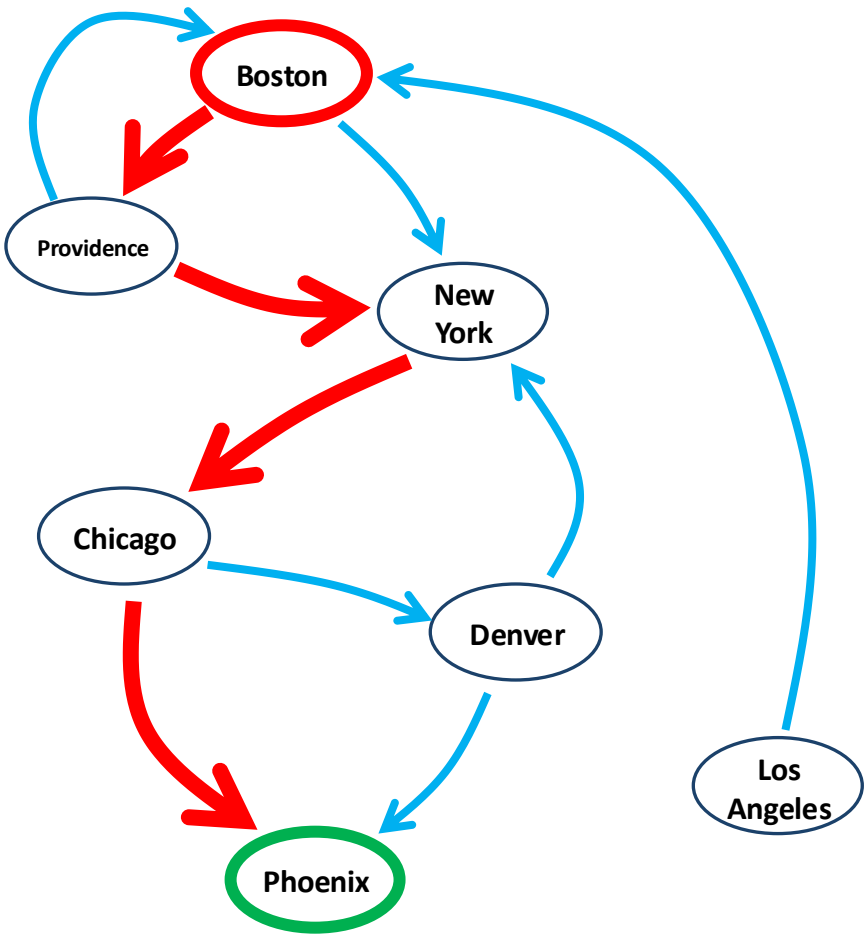
Moving from Trees to General Graphs

Challenge 1:
Multiple possible paths from start to destination



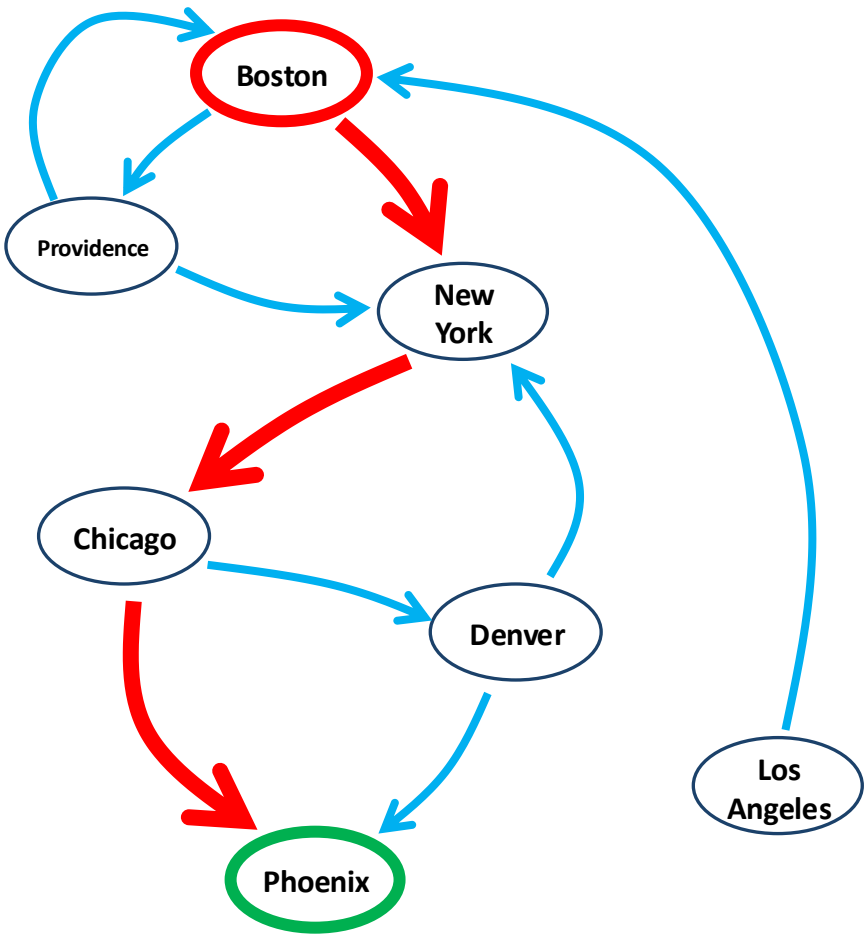
Graph Issues

Challenge 1:
Multiple possible paths from start to destination



Graph Issues

Challenge 1:
Multiple possible paths from start to destination

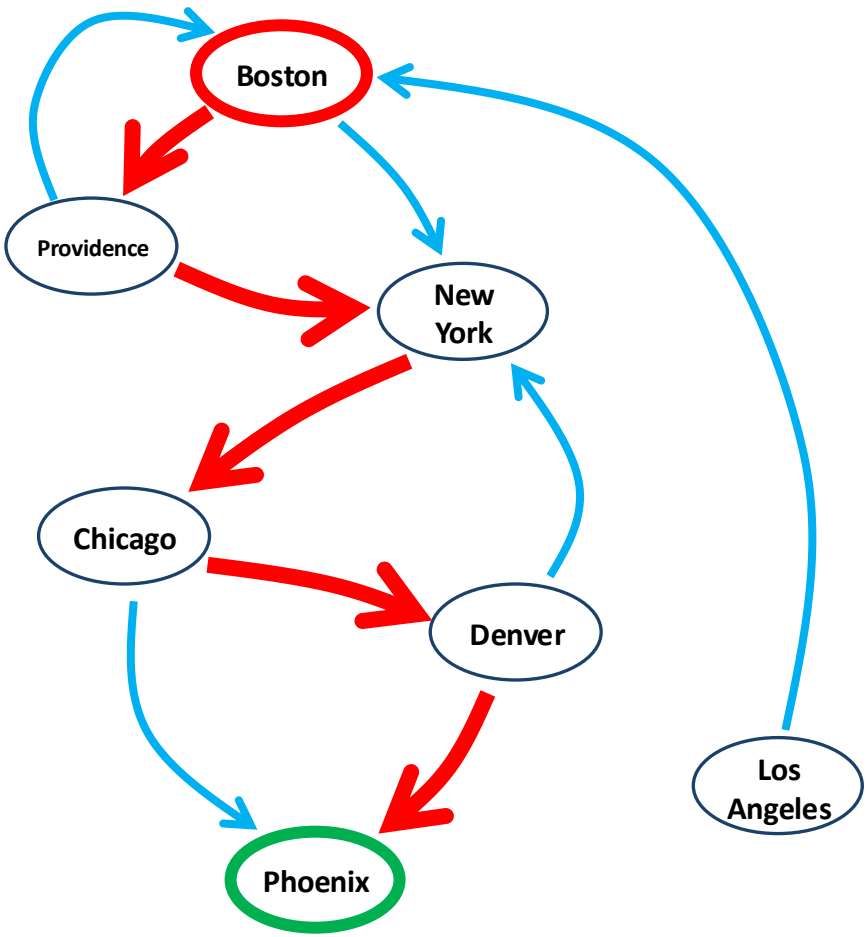


Graph Issues

Challenge 1: Multiple possible paths from start to destination.

For BFS first path found is always a shortest path, because paths explored in order of length.

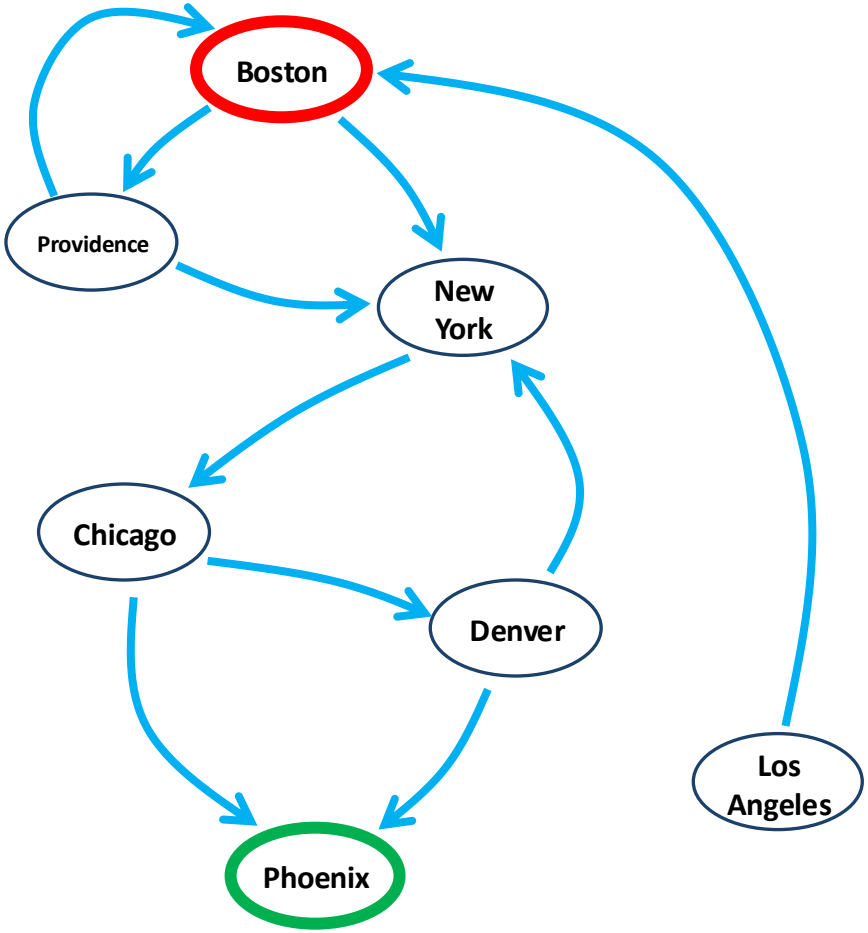
For DFS first path found is **not** always a shortest path, so will have to change it.



Graph Issues

Challenge 1:
Multiple possible paths from start to destination

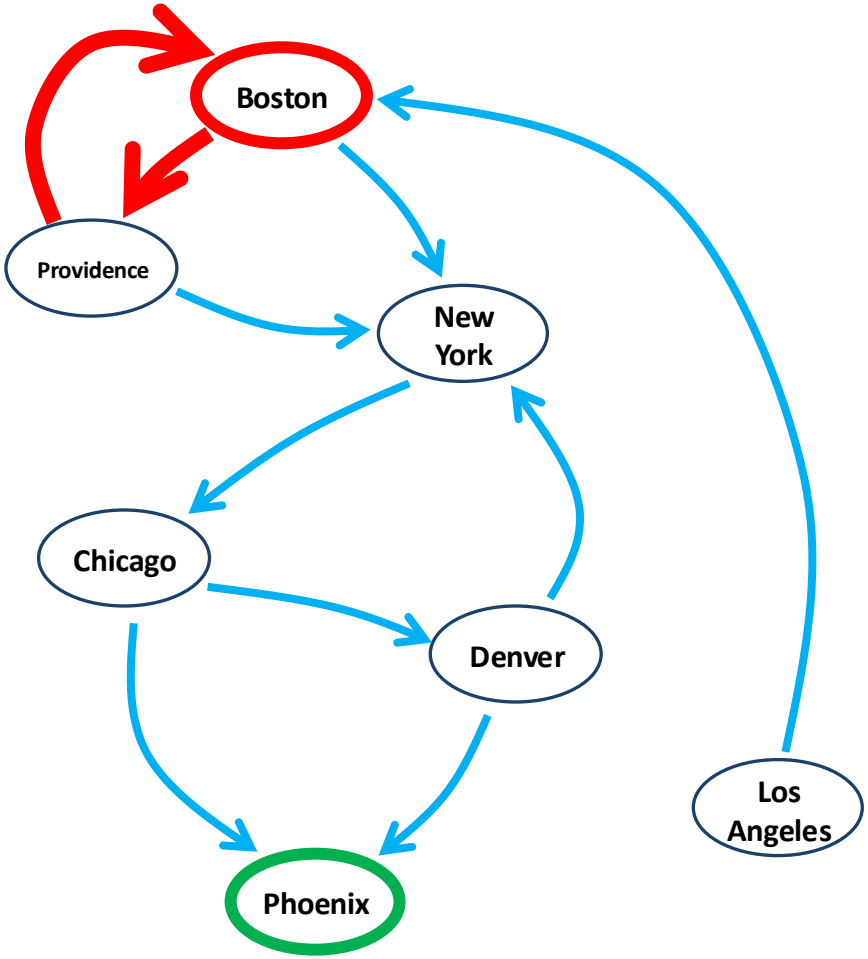
Challenge 2:
Need to avoid infinite cycles



Graph Issues

Challenge 1:
Multiple possible paths from start to destination

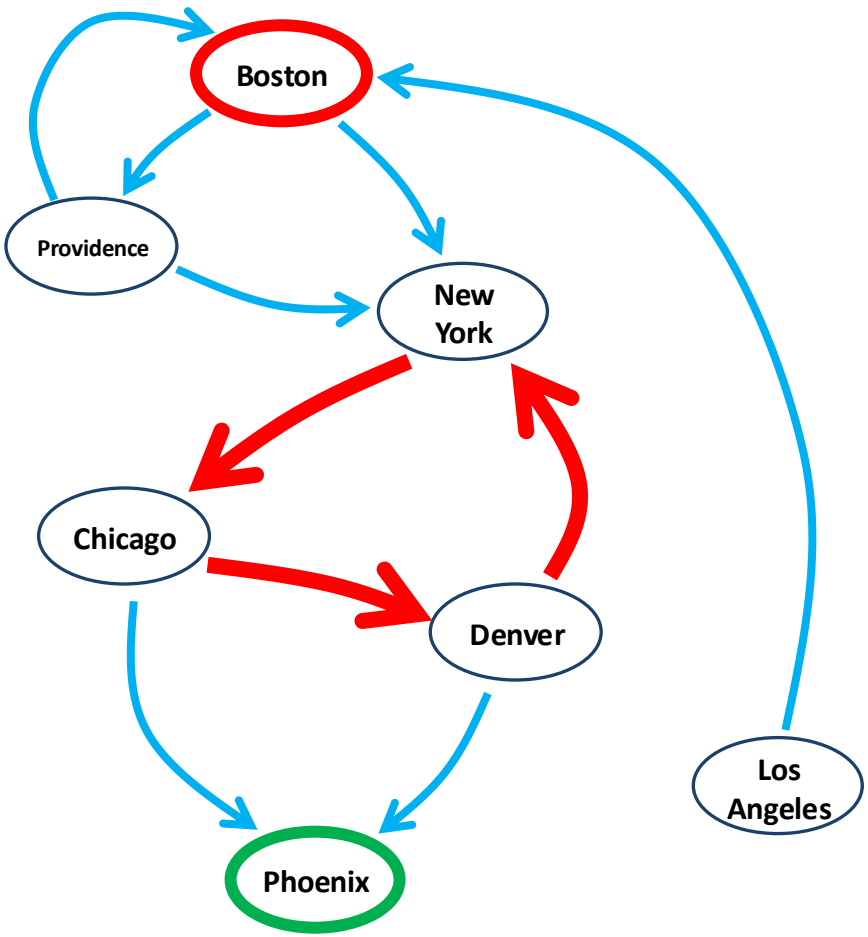
Challenge 2:
Need to deal with cycles



Graph Issues

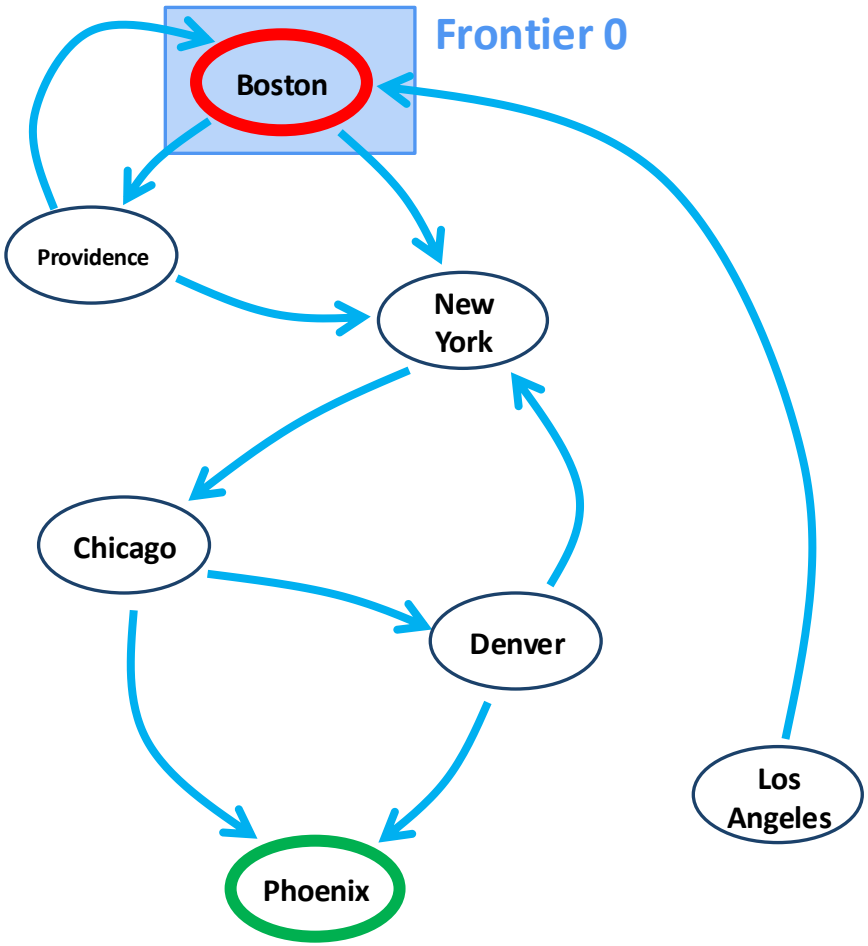
Challenge 1:
Multiple possible paths from start to destination

Challenge 2:
Need to avoid infinite cycles



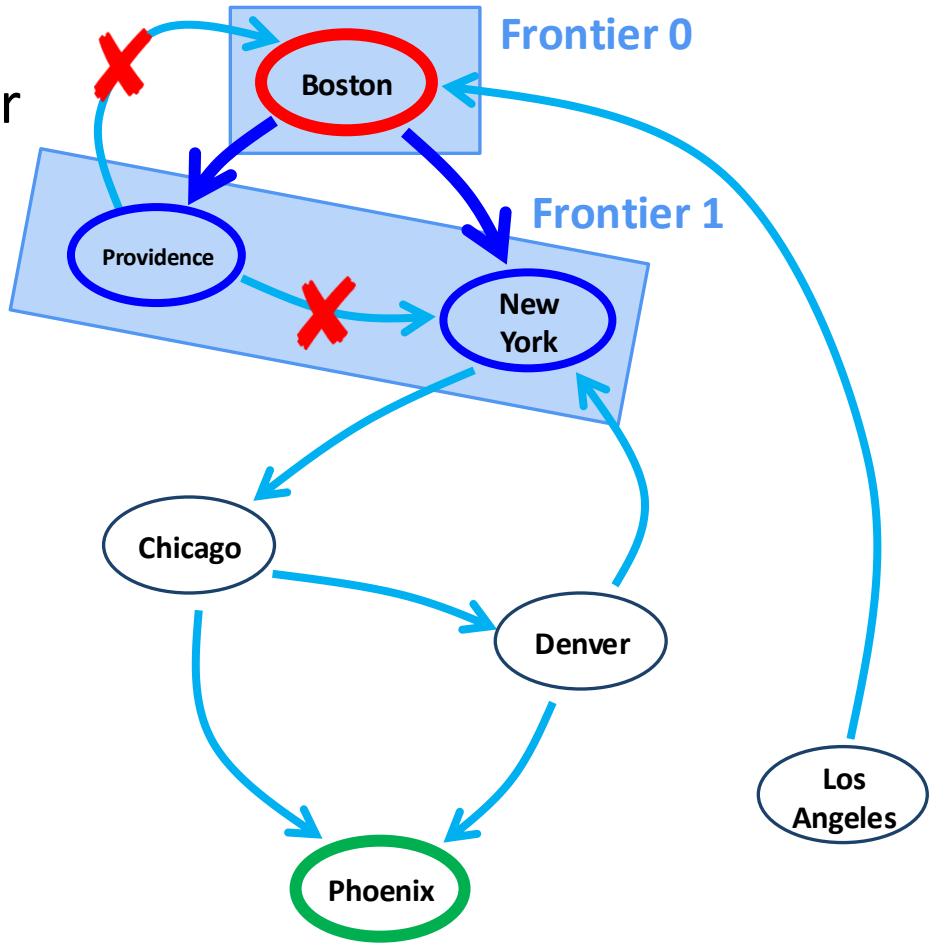
BFS: Avoiding Cycles

- Already **storing paths** so far when expanding their end nodes
- Reject any new child already in a **previous frontier**



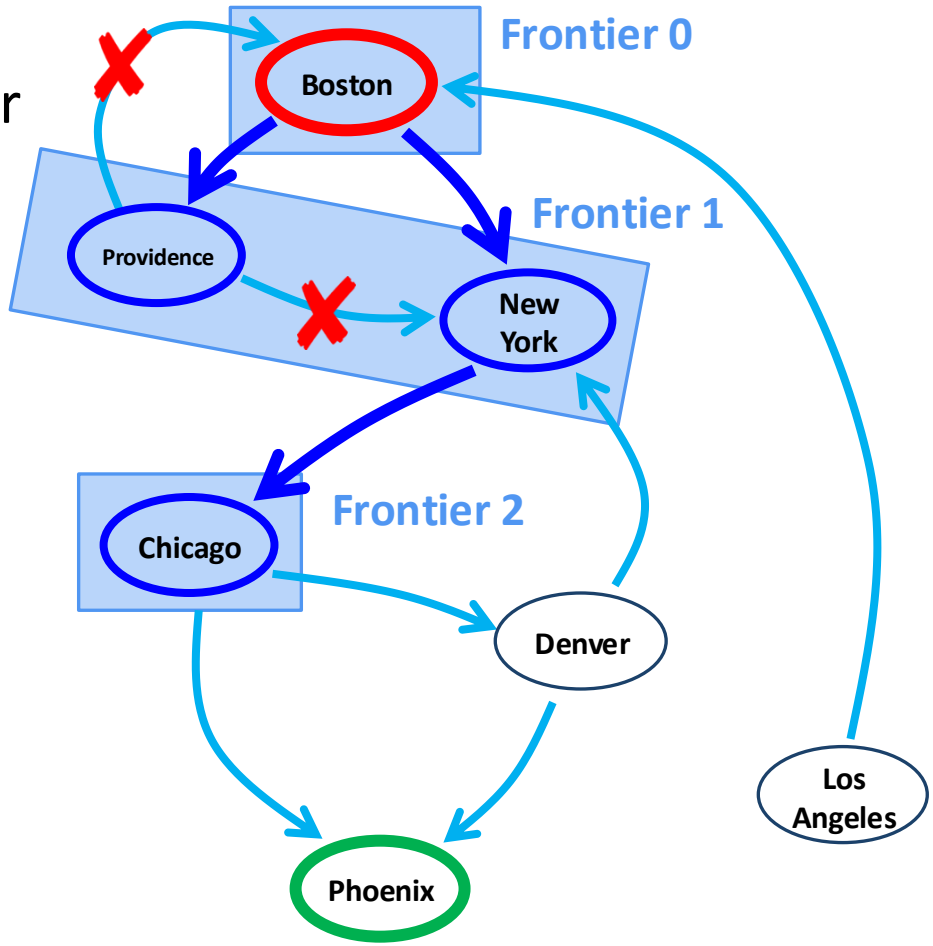
BFS: Avoiding Cycles

- Already **storing paths** so far when expanding their end nodes
- Can do even better: Reject any new child already in a **previous frontier**



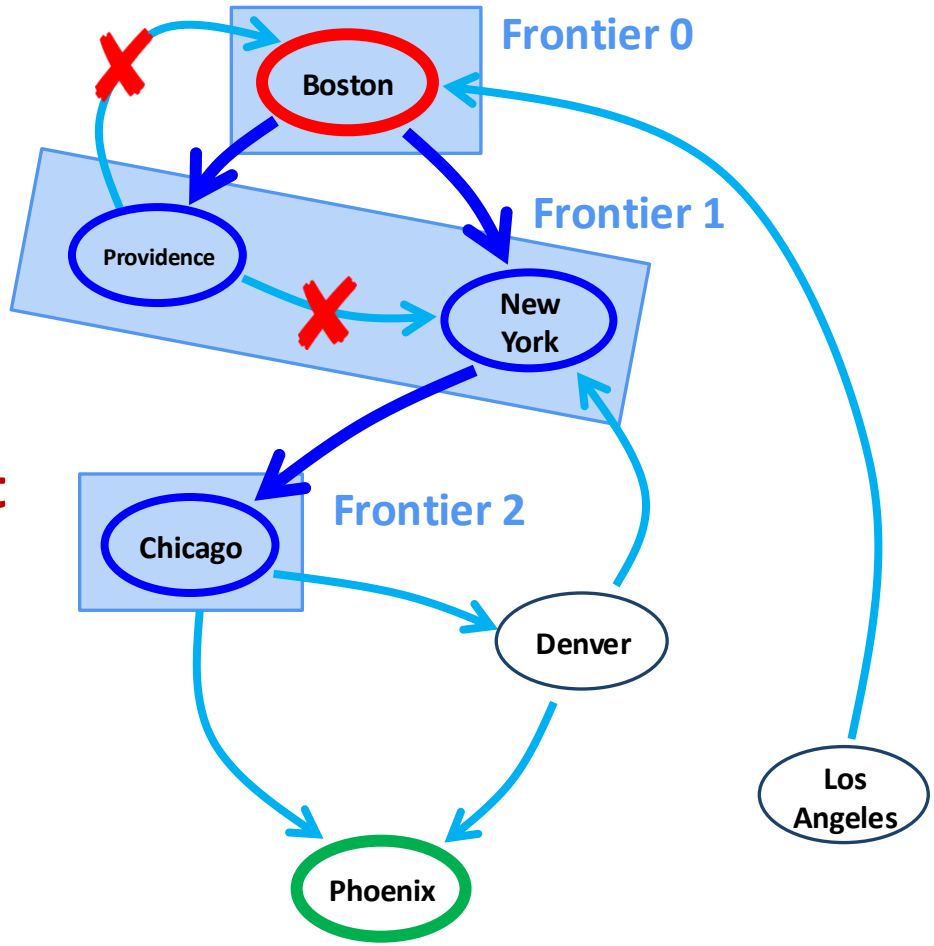
BFS: Avoiding Infinite Cycles

- Already **storing paths** so far when expanding their end nodes
- Can do even better: Reject any new child already in a **previous frontier**



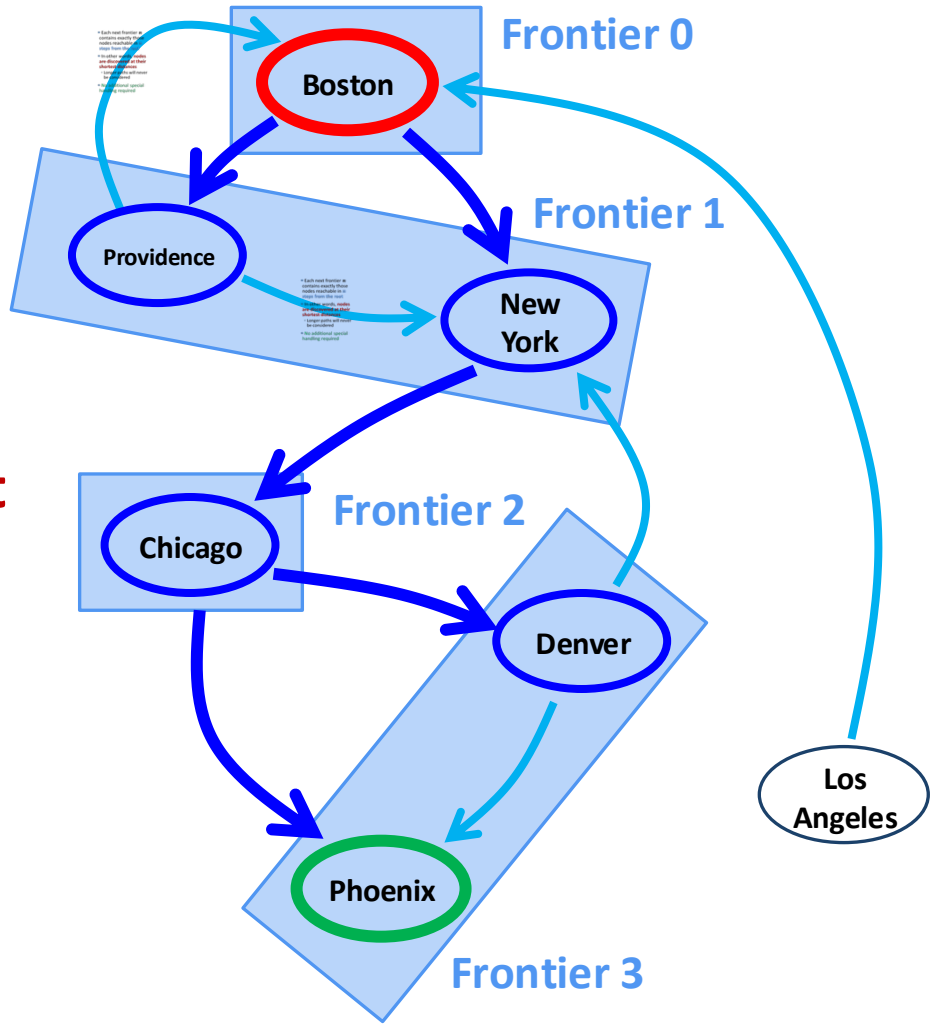
BFS: Handling Multiple Paths

- Each next frontier n contains exactly those nodes reachable in n steps from the root
- In other words, **nodes are discovered at their shortest distances**
  - Longer paths will never be considered



BFS: Handling Multiple Paths

- Each next frontier n contains exactly those nodes reachable in n steps from the root
- In other words, **nodes are discovered at their shortest distances**
  - Longer paths will never be considered
- No additional special handling required



BFS on Graphs

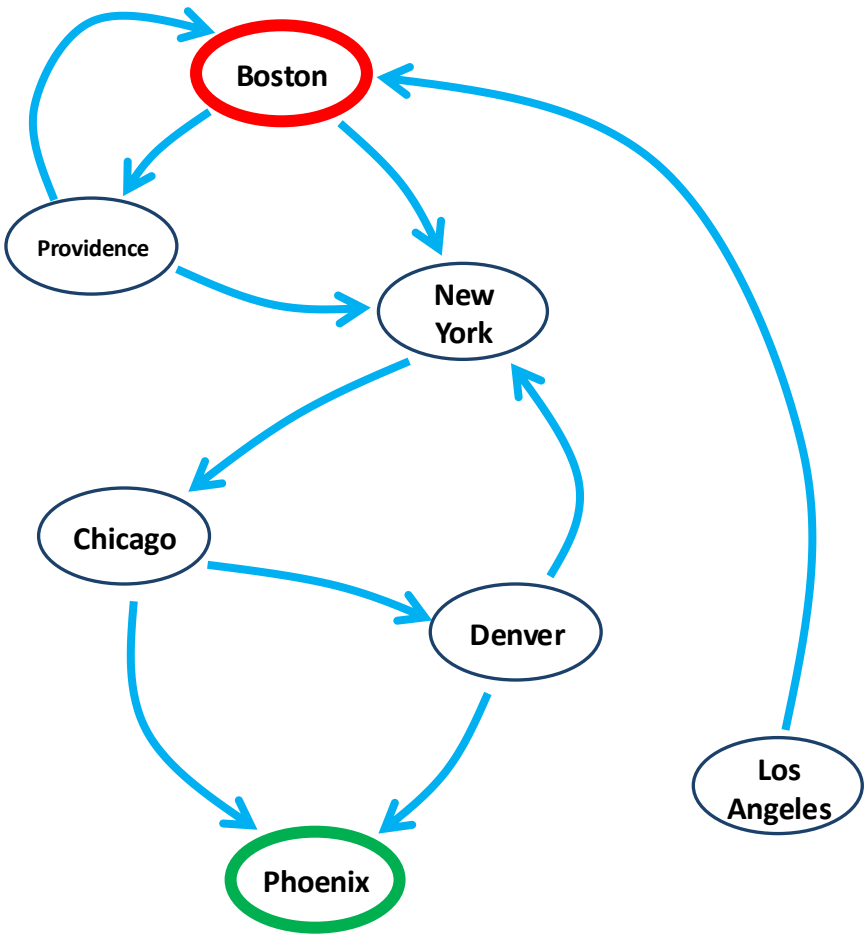
```
def bfs_graph(graph, source, target):
    current_frontier = [[source]]
    next_frontier = []
    visited = set()
    visited.add(source)

    while current_frontier:
        for path in current_frontier:
            last_node = path[-1]
            for next_node in graph.children_of(last_node):
                if next_node in visited:
                    continue
                new_path = path + [next_node]
                visited.add(next_node)
                if next_node == target:
                    return new_path
                next_frontier.append(new_path)

        current_frontier, next_frontier = next_frontier, []
    return None
```

Maintain a visited set to avoid going in cycles

Still return first path found to target

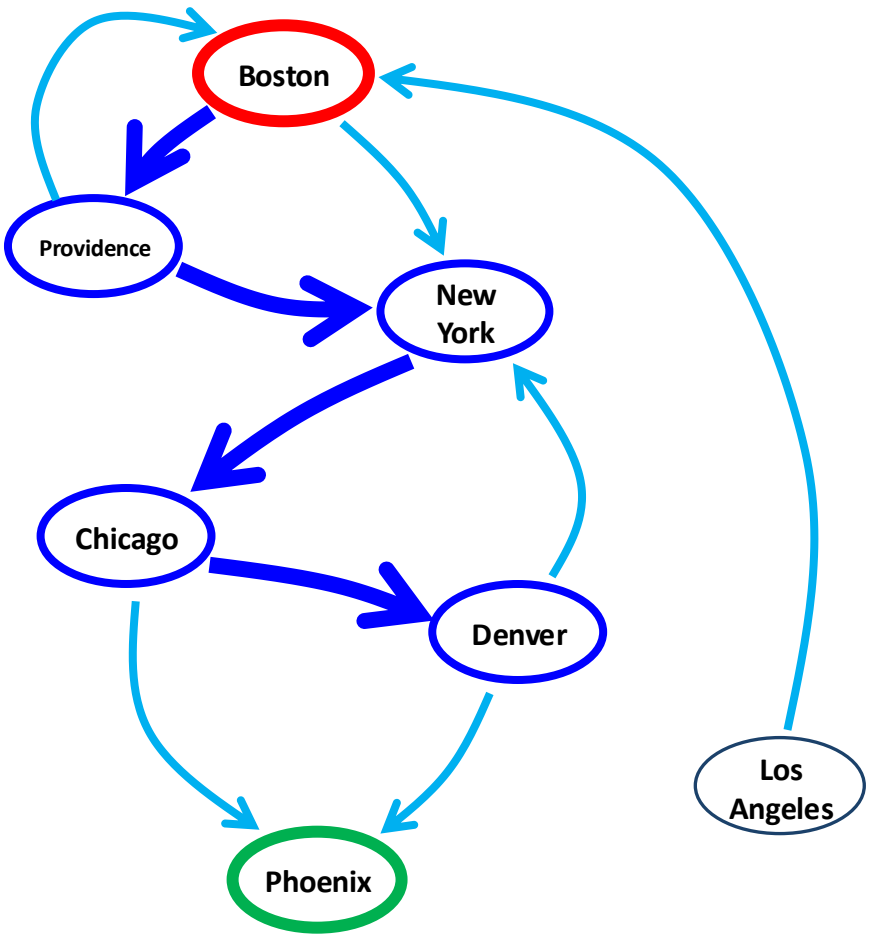
DFS: Avoiding Cycles

- Already **storing paths** so far when expanding their end nodes



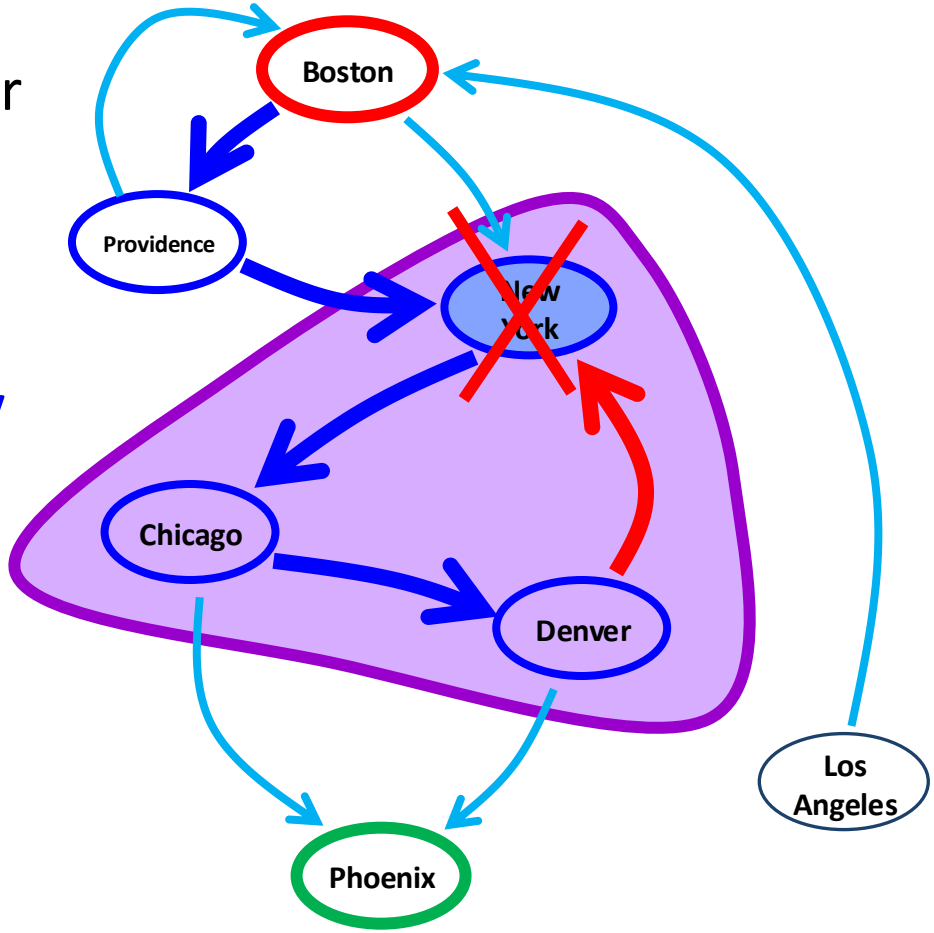
DFS: Avoiding Cycles

- Already **storing paths** so far when expanding their end nodes
- Encounter cycles when expanding to **nodes already on the path**



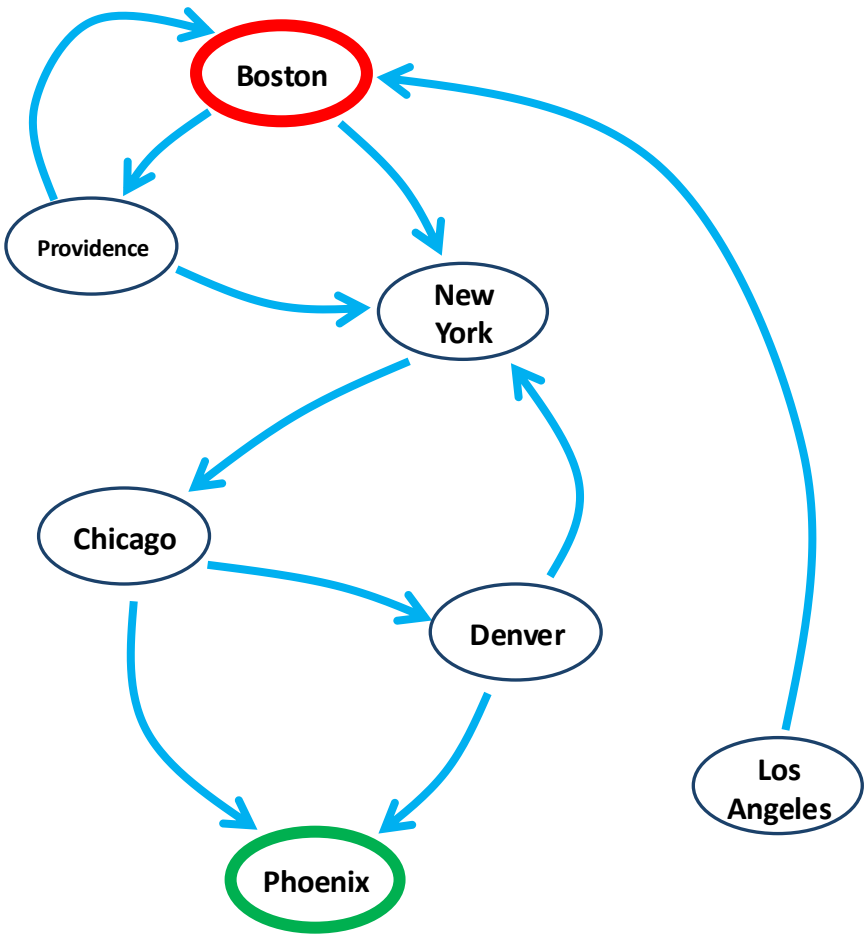
DFS: Avoiding Cycles

- Already **storing paths** so far when expanding their end nodes
- Encounter cycles when expanding to **nodes already on the path**
- Prevent **DFS "subtrees"** from looping back on themselves



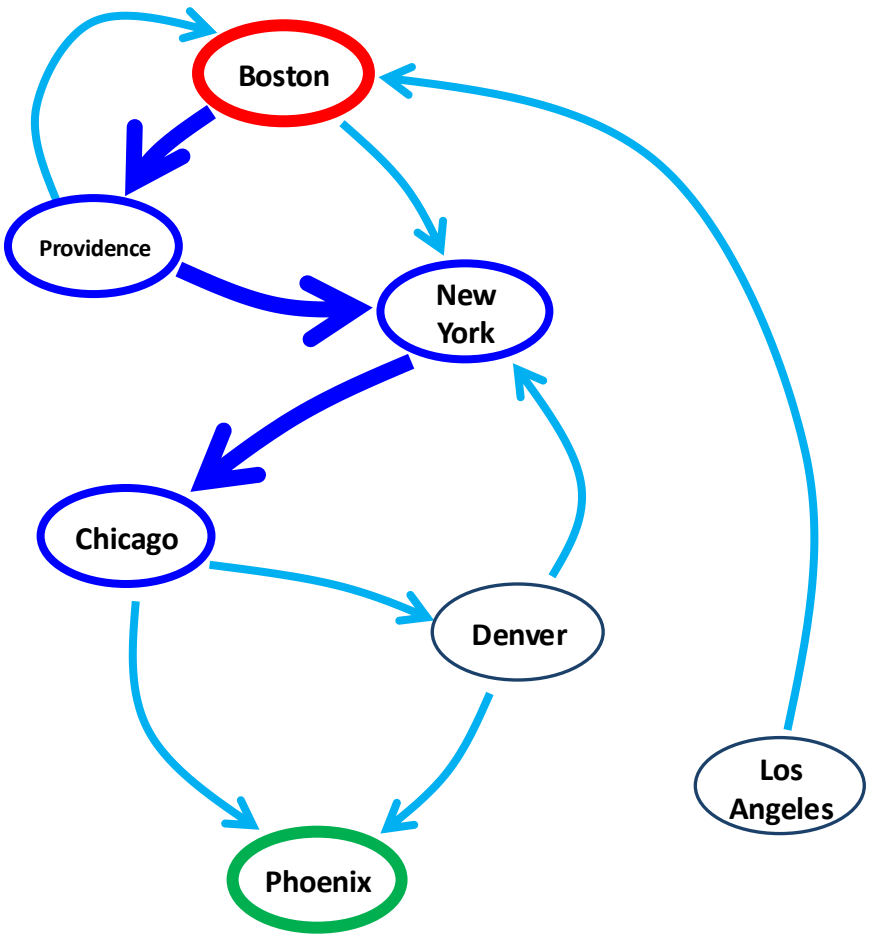
DFS: Considering Multiple Paths

- When recursive calls of a node's children return paths to target, **pick the shortest one**



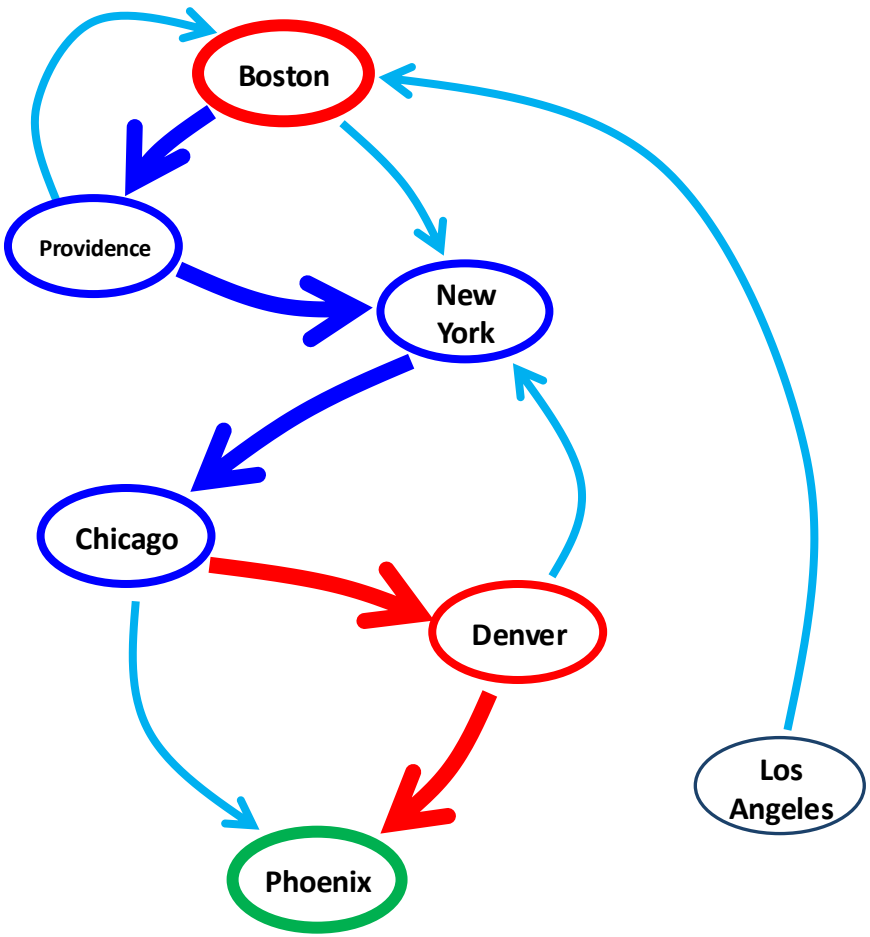
DFS: Considering Multiple Paths

- When recursive calls of a node's children return paths to target, **pick the shortest one**



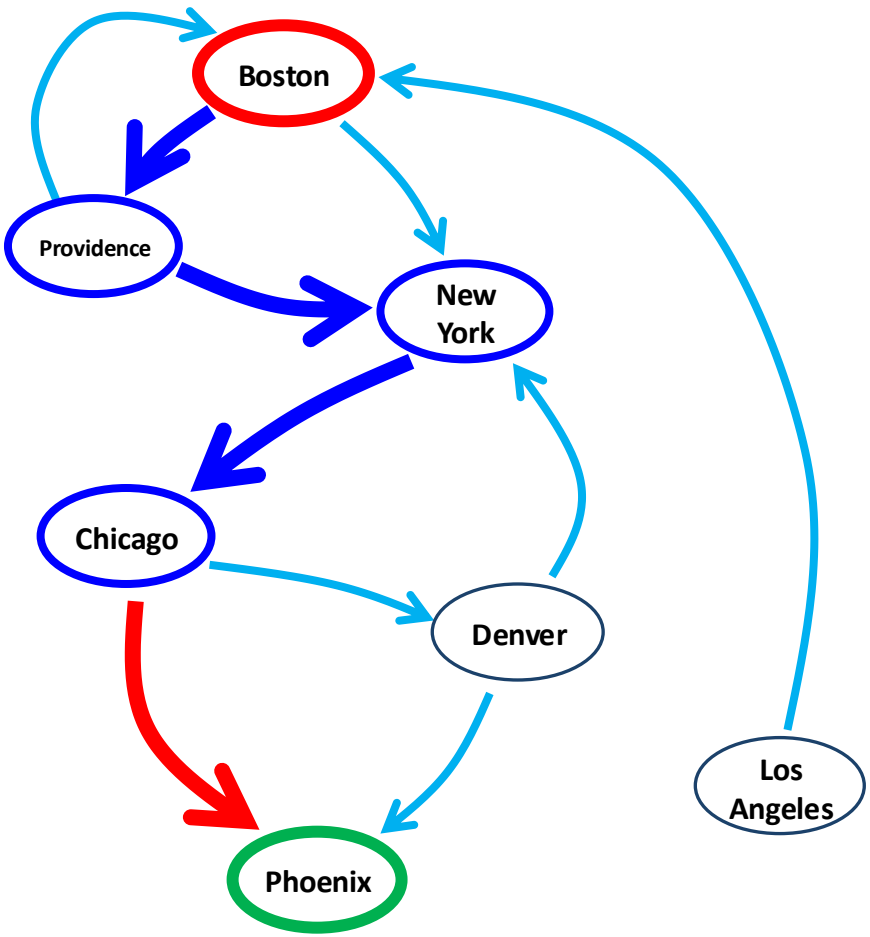
DFS: Considering Multiple Paths

- When recursive calls of a node's children return paths to target, **pick the shortest one**



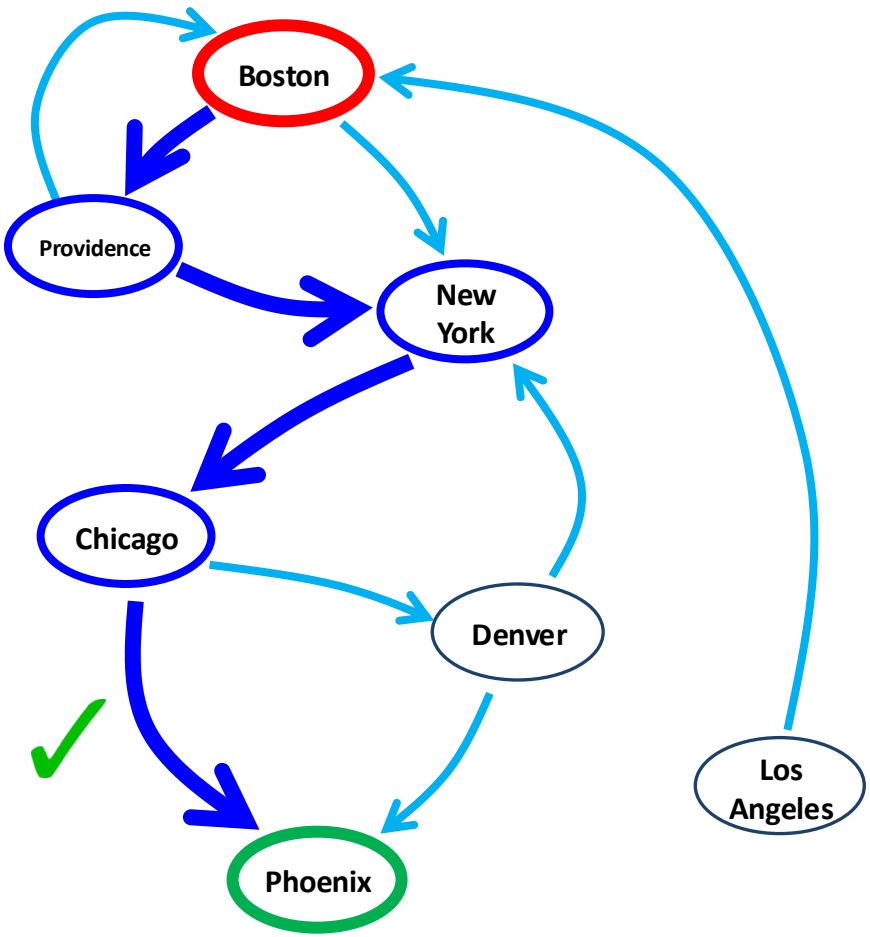
DFS: Considering Multiple Paths

- When recursive calls of a node's children return paths to target, **pick the shortest one**



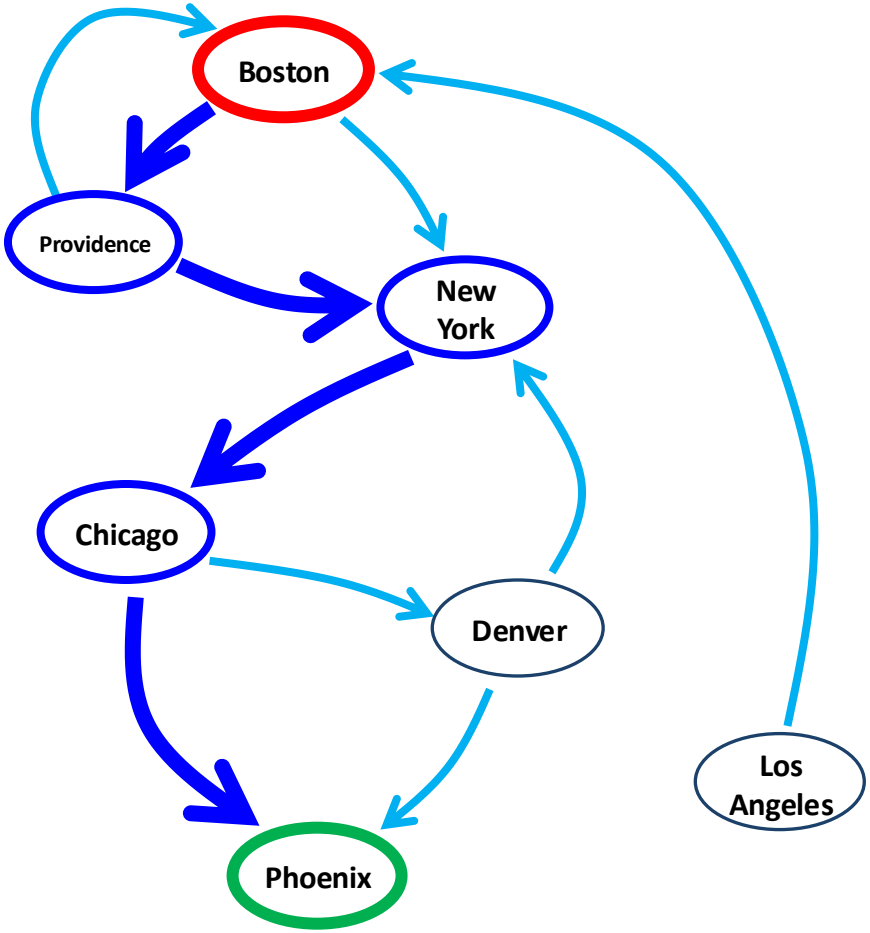
DFS: Considering Multiple Paths

- When recursive calls of a node's children return paths to target, **pick the shortest one**



DFS: Considering Multiple Paths

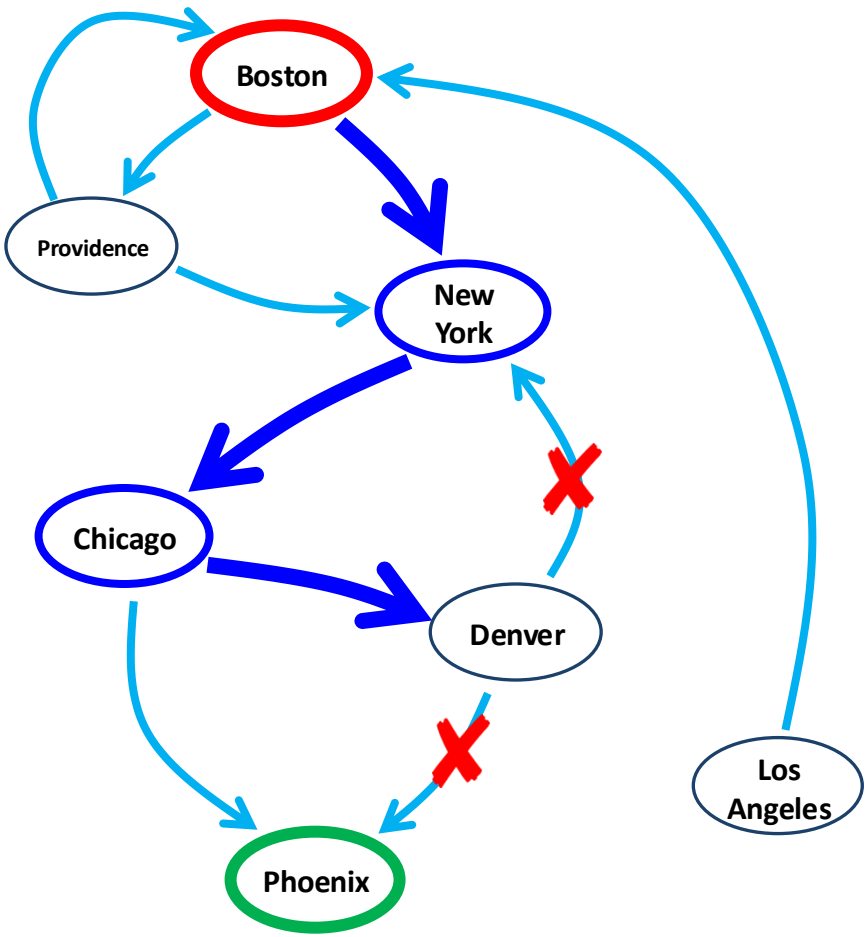
- When recursive calls of a node's children return paths to target, **pick the shortest one**
- Maintain a **shortest path yet found**, so **subsequent DFS subtrees** don't have to explore any deeper



shortest

DFS: Considering Multiple Paths

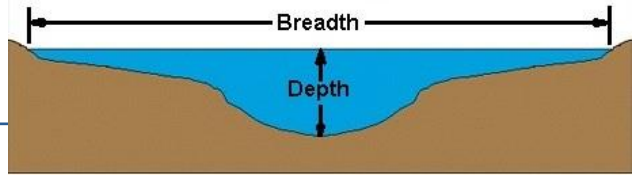
- When recursive calls of a node's children return paths to target, **pick the shortest one**
- Maintain a **shortest path yet found**, so **subsequent DFS subtrees** don't have to explore any deeper
- Similar to trick we used for knapsack problem



Let's Try It

```
Current DFS path: Boston
Current DFS path: Boston->Providence
Current DFS path: Boston->Providence->New York
Current DFS path: Boston->Providence->New York->Chicago
Current DFS path: Boston->Providence->New York->Chicago->Denver
Current DFS path: Boston->Providence->New York->Chicago->Denver->Phoenix
Path found of length 6
Current DFS path: Boston->Providence->New York->Chicago->Phoenix
Path found of length 5
Current DFS path: Boston->New York
Current DFS path: Boston->New York->Chicago
Current DFS path: Boston->New York->Chicago->Denver
Current DFS path: Boston->New York->Chicago->Phoenix
Path found of length 4
['Boston', 'New York', 'Chicago', 'Phoenix']
```

Comparing BFS and DFS



Breadth-first search

- Can rely on visited set ✓
  - Each node in it already has shortest path from source
  - Considers way fewer paths
- Has to store **all** paths of length ***n*** before considering paths of length ***n+1*** ✗

Could be memory-intensive

Depth-first search

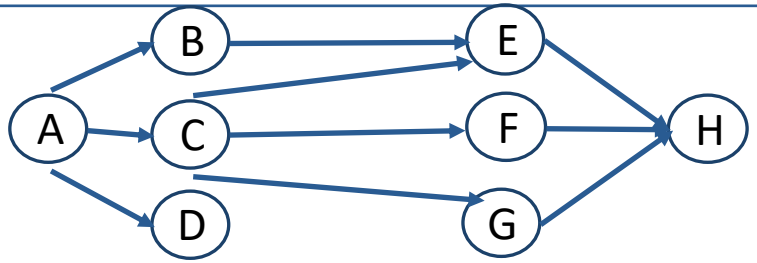
- Has to consider multiple paths to same nodes ✗
  - Subsequent paths might be shorter
- Won't explore any paths longer than shortest path found so far
  - Might get lucky and find a reasonable path early on ✓
  - Might get unlucky and waste time exploring unpromising chunks of graph ✗

A Different Way of Looking at DFS and BFS

- Last-in first-out (LIFO) sequence (often called a stack)
- First-in first-out (FIFO) sequence (often called a queue

FIFO and BFS Shortest Path

Seeking path from A to H



Take next path from front, and delete front. Add new paths to rear.

Guarantees first solution is a shortest path

queue	tmp_path	
[[A]]	A	
[[A,B],[A,C],[A,D]]	A→B	Added paths from A
[[A,C],[A,D],[A,B,E]]	A→C	Added paths from B
[[A,D],[A,B,E],[A,C,F],[A,C,G]]	A→D	Added paths from C, don't revisit E
[[A,B,E],[A,C,F],[A,C,G]]	A→B→E	No paths from D
[[A,C,F],[A,C,G],[A,B,E,H]]		Added paths from E

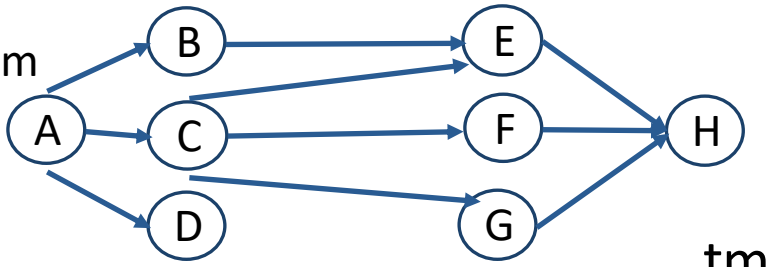
Found a path

Might be other **equally** short paths

But don't care

LIFO and DFS Shortest Path

Seeking path from A to H



Take next path from rear, and delete rear. Add new paths to rear.

stack	tmpPath	
[[A]]	A	
[[A,B],[A,C],[A,D]]	A→D	Added paths from A
[[A,B],[A,C]]	A→C	No paths from D
[[A,B],[A,C,E],[A,C,F],[A,C,G]]	A→C→G	Added paths from C
[[A,B],[A,C,E],[A,C,F],[A,C,G,H]]	A→C→G→H	Added paths from G
[[A,B],[A,C,E],[A,C,F]]	No need to explore further	
[[A,B],[A,C,E]]	A→C→E	
	No need to explore further	
[[A,B]]	A →B	
[[A,B,E]]	A→B→E	
	No need to explore further	

BFS Using a FIFO Queue

```
def bfs_fifo(graph, source, target):
    stack = [[source]]
    visited = set()
    visited.add(source)

    while stack:
        path = stack.pop(0)

        last_node = path[-1]
        for next_node in graph.children_of(last_node):
            if next_node in visited:
                continue
            new_path = path + [next_node]
            visited.add(next_node)
            if next_node == target:
                return new_path
            stack.append(new_path)

    return None
```

Replace discrete frontiers with "sliding" stack

FIFO

DFS Using a LIFO Stack

```
def dfs_lifo(graph, source, target):
    stack = [[source]]
    best_path = None
    while stack:
        path = stack.pop(-1)

        if best_path and len(path) + 1 >= len(best_path):
            continue

        last_node = path[-1]
        for next_node in graph.children_of(last_node):
            if next_node in path:
                continue
            new_path = path + [next_node]
            if next_node == target:
                if not best_path or len(new_path) < len(best_path):
                    best_path = new_path
                continue
            stack.append(new_path)
    return best_path
```

Simulate recursion with expanding/shrinking stack

LIFO

No need to maintain a separate **shortest** path

Put children on LIFO stack

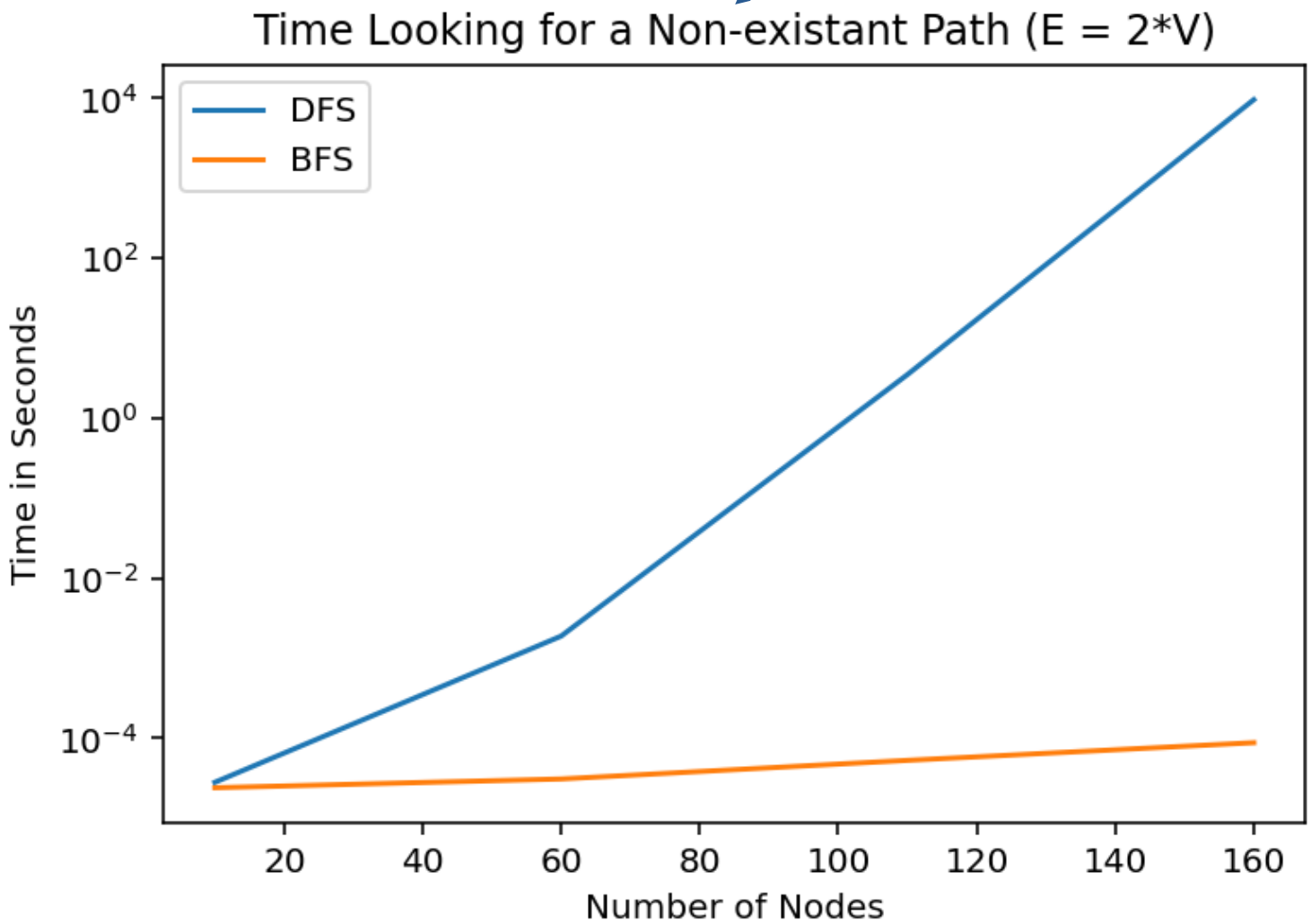
Keep iterating instead of returning from recursion

Why Am I Showing You This?

- It's kind of cute
- Shows you how to implement a recursive algorithm without recursion
  - Usually far more efficient than recursion
- Needed to implement it so that I could produce a plot I want to show you. Recursive version of DFS took too long.
  - Will generate a large random graph
  - Examine worst case time for DFS and BFS

Worst Case Speed Is an Issue

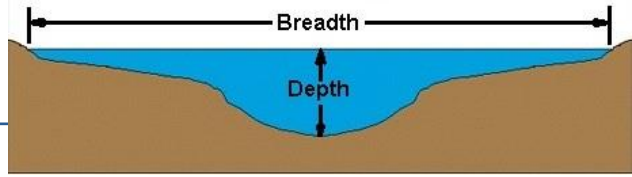
Why?



About 158 minutes

Way less than a second

Comparing BFS and DFS



Breadth-first search

- Can rely on visited set
  - Each node in it already has shortest path from source
  - Considers way fewer paths
- Has to store **all** paths of length ***n*** before considering paths of length ***n+1***

Could be memory-intensive

Depth-first search

- Has to consider multiple paths to same nodes
  - Subsequent paths might be shorter
- Won't explore any paths longer than shortest path found so far
  - Might get lucky and find a reasonable path early on
  - Might get unlucky and waste time exploring unpromising chunks of graph

Summary

- We use graphs to model situations in which the relationships among things are important
  - These relationships might be spatial, but need not be
- Many problems are related to path finding in a graph
  - Shortest paths often of particular interest
- Depth-first and breadth-first search are simple ways to find paths in graphs
  - Breadth-first seems much faster, but there is a caveat we will see in the the next lecture
- Next lecture we will look at
  - Weighted shortest path
  - More sophisticated path-finding algorithms
  - Other graph problems